

嵌入式系统

第四讲 ARM硬件结构（二）

教师：郭玉臣

Mail:yuchenguo@tongji.edu.cn



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

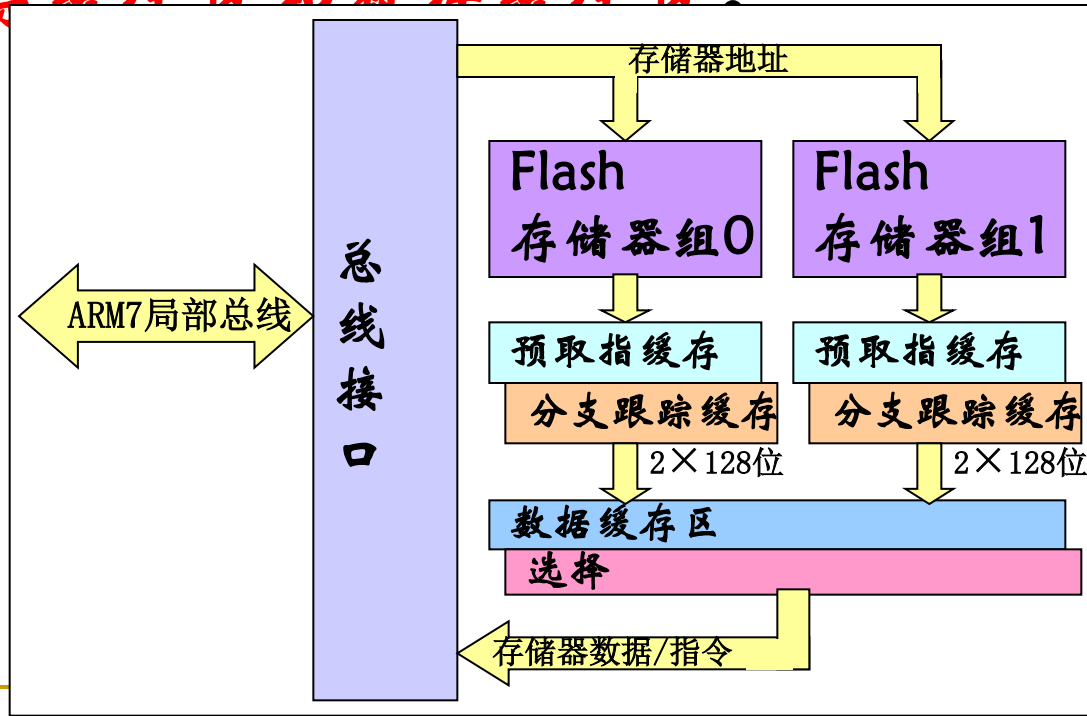
4.5 存储器加速模块 (MAM)

- 概述

LPC2000微控制器扩展了器件内部Flash总线宽度为128位，用于提高处理器的指令执行速度。这个接口通过存储器加速模块(MAM)来控制。

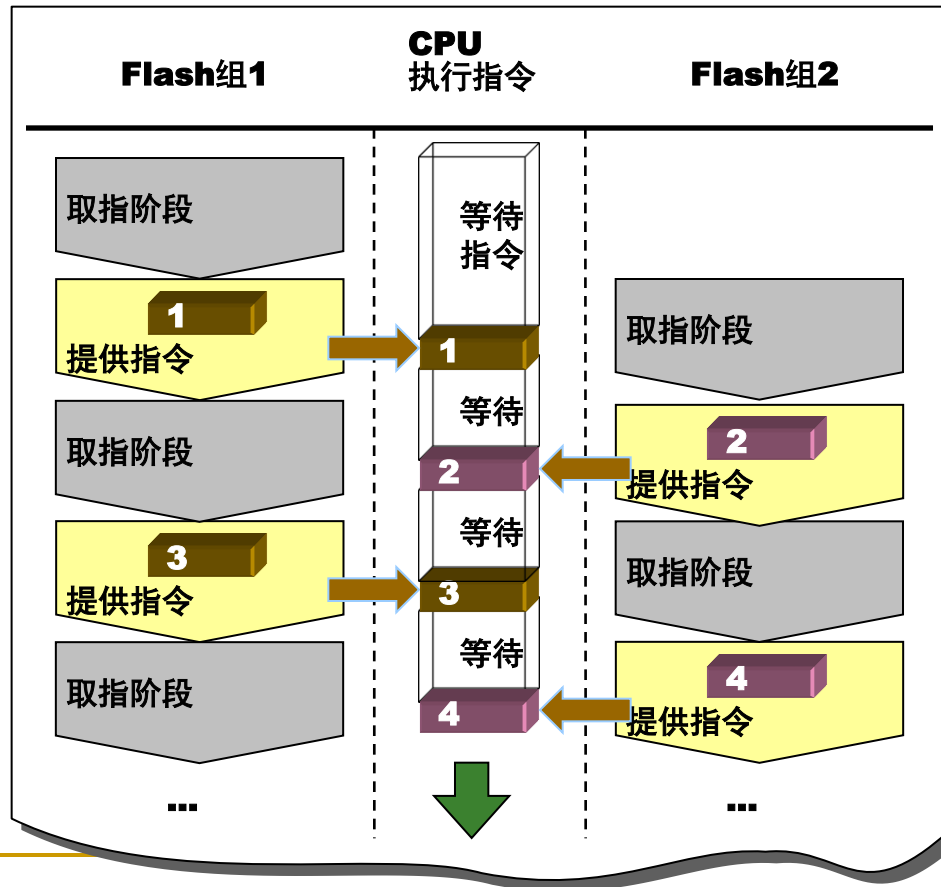
• MAM内部结构

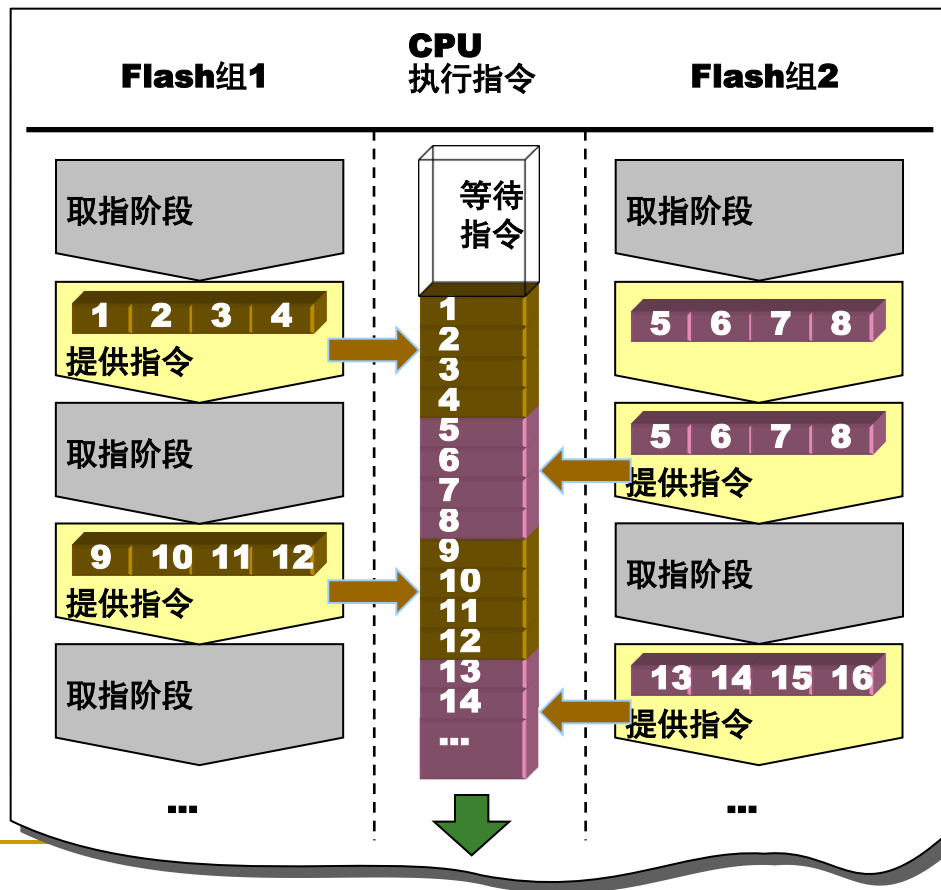
LPC2200系列芯片把Flash存储器被分成两组，它们轮番工作，及时的为CPU提供需要的指令和数据，以防止CPU取指暂停。每组Flash存储器都有自己的预取指缓冲区、分支跟踪缓冲区和数据缓冲区。



关闭MAM指令执行情况

4 CPU在Flash组1提供指令时，Flash组2正在等待指令，由此产生了CPU停止，等待若干周期的情况。





4.5 存储器加速模块 (MAM)

- 程序顺序执行

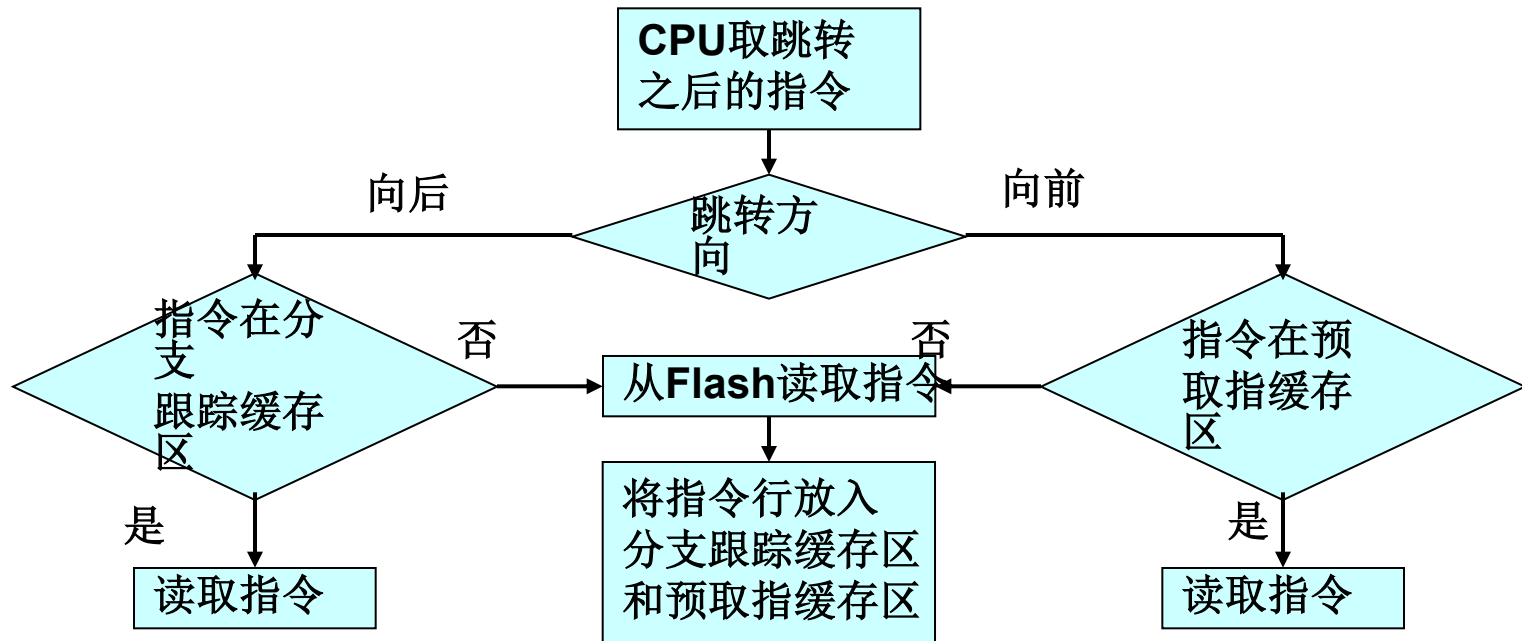
每个128位值包括了4个32位ARM指令或8个16位Thumb指令。

在连续执行代码时，通常一个Flash组包含当前正在取指的指令和包含该指令的整个Flash行，而另一个Flash组则包含或正在预取指下一个连续的代码行。

4.5 存储器加速模块（MAM）

- 程序出现分支

在预取指缓存区中保存着将要执行的指令，在分支跟踪缓存区中保存着程序跳转后可执行到的指令。



4.5 存储器加速模块（MAM）

- 程序获取数据

当CPU在从Flash中获取数据时，如果这些数据没有出现在MAM的数据缓冲区中，那么MAM会执行一次Flash读操作，并把一个128位的数据行存入数据缓冲区，这样就加快了按顺序访问数据的速度。数据访问使用一个单行的缓冲区，和访问代码时提供两个缓冲区不同，因为数据访问不需要预取指功能。

4.5 存储器加速模块（MAM）

- **MAM与Flash编程**

Flash编程功能不受存储器加速器模块的控制，而是作为一个独立的功能进行处理。Flash存储器的布线使其每个扇区同时存在于两个组当中，这样扇区擦除操作可同时对两个组执行。

4.5 存储器加速模块（MAM）

- MAM的操作模式

LPC2000系列芯片允许用户设置MAM的加速级别，使芯片适用于某些对功耗和可预测性有要求的场合。MAM定义了三种操作模式：

加速级别	顺序执行	程序分支	数据	功耗	可预测性
关闭	不预取代码	不缓冲代码	不缓冲数据	低	高
部分使能	预取代码	缓冲代码	缓冲但时序固定	中	中
完全使能	预取代码	缓冲代码	缓冲数据	高	低



4.5 存储器加速模块（MAM）

- MAM的配置

在复位后，MAM默认为禁止状态。软件可以随时将存储器访问加速打开或关闭。通常我们都会把加速设置为完全使能，这可使程序以最高速度运行。而运行某些要求更精确定时的代码时，我们可以关闭或部分使能MAM，以较慢但可预测的速度运行代码。

4.5 存储器加速模块 (MAM)

• 寄存器描述

MAM控制寄存器(MAMTCR):

MAMTIM[2:0]: 决定使用多少个CCLK周期访问Flash存储器, 这样可以调整MAM时序使其匹配处理器操作频率, 具体关系如下表。

MAMTIM[2:0]	描述	MAMTIM[2:0]	描述
000	保留	100	取指周期为4个CCLK
001	取指周期为1个CCLK	101	取指周期为5个CCLK
010	取指周期为2个CCLK	110	取指周期为6个CCLK
011	取指周期为3个CCLK	111	取指周期为7个CCLK

当访问周期设置为1个CCLK时, 实际上关闭了MAM, 但是仍可控制MAM模式对功耗进行优化。

MAMTIM

2:0

取指周期控制

注意: 不正确的设定会导致器件的错误操作

4.5 存储器加速模块 (MAM)

- MAM使用注意事项

MAM编程问题:

在编程时，MAM定时值过程中须先关闭MAMFlash存储器，否则会导致Flash存储器中读取失败的数据，导致Flash编程或擦除操作失败。MAM将不缓冲导致操作错误。所以在Flash操作结束后，任何对Flash地址的读操作都将启动新的取指操作。

系统时钟(CCLK)	MAM定时值
< 20MHz	1
20MHz ~ 40MHz	2
40MHz ~ 60MHz	3

4.5 存储器加速模块（MAM）

• MAM应用示例

MAM部件Flash访问时钟的配置：

- ❶ 关闭MAM部件；
- ❷ 根据系统时钟配置取指周期；
- ❸ 启动MAM部件。

```
void MAMSet(uint32 Fcclk)
{
    MAMCR = 0;
    If(Fcclk < 200000000)
    {
        MAMTIM = 1;
    }
    else
    if(Fcclk < 400000000)
    {
        MAMTIM = 2;
    }
    else
    {
        MAMTIM = 3;
    }
    MAMCR = 2;
}
```



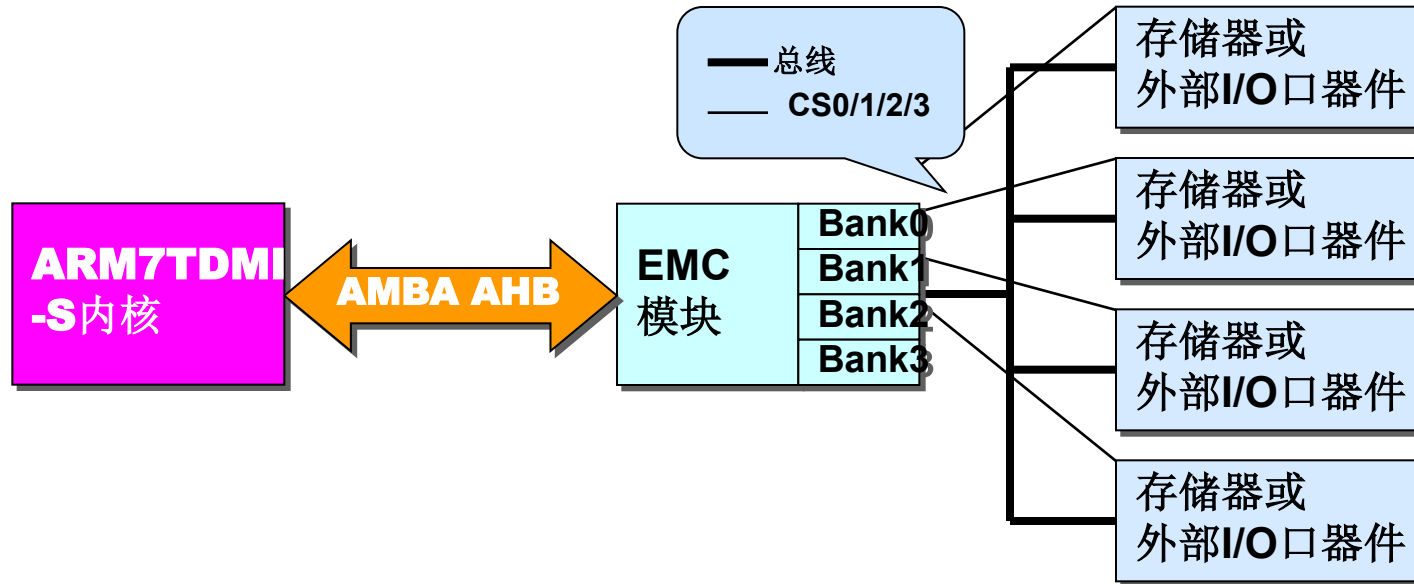
LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.6 外部存储器控制器（EMC）

概述

该模块将系统总线（如AMBA AHB）与外部存储器（如SRAM、NOR Flash、NAND Flash、DDR SDRAM等）或外部I/O器件（如SD卡、USB、以太网控制器等）连接起来。它通常支持多种存储器接口，并可提供片内缓存（如SRAM、NOR Flash、NAND Flash、DDR SDRAM等）或外部I/O器件。



4.6 外部存储器控制器（EMC）

• 特性

- 支持静态存储器映射器件；
- 4个存储器组(Bank0 ~ Bank3)可单独配置，每个存储器组可访问16M字节空间；
- 总线空闲周期可编程；
- 可对静态RAM器件的读写等待时间进行编程；
- 可编程外部数据总线宽度——8位、16位和32位。



4.6 外部存储器控制器（EMC）

• 引脚描述

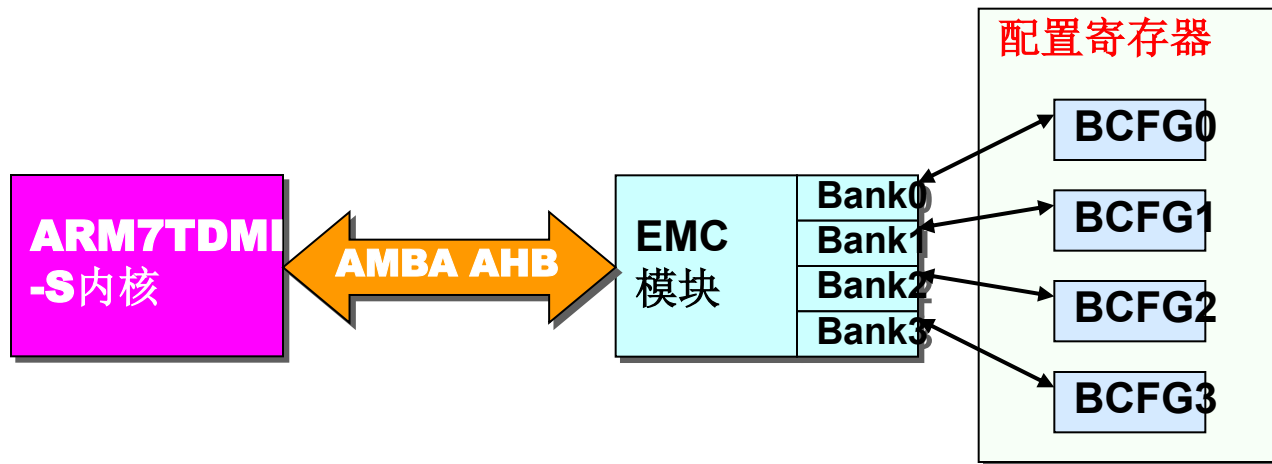
引脚名称	类型	引脚描述	复用引脚
D[31:0]	输入/输出	外部存储器数据线	P2.0~P2.31
A[23:0]	输出	外部存储器地址线	P3.0~P3.23
OE	输出	读使能信号，低有效	P1.1
WE	输出	写使能信号，低有效	P3.27
CS[3:0]	输出	片选信号，低有效	P1.0、P3.24~P3.26
BLS[3:0]	输出	字节定位选择信号，低有效	P3.28~P3.31

注：每个这些引脚是与P1、P2和P3口GPIO功能复用，所以在使用外部总线前首先要正确配置PINSEL2寄存器。

4.6 外部存储器控制器（EMC）

• 寄存器描述

外部存储器控制器包含4个配置寄存器。每个配置寄存器控制一个外部存储器接口的**总线空闲时间、读访问长度、写访问长度、存储器组是否写保护、存储器组的总线宽度的设置等**。



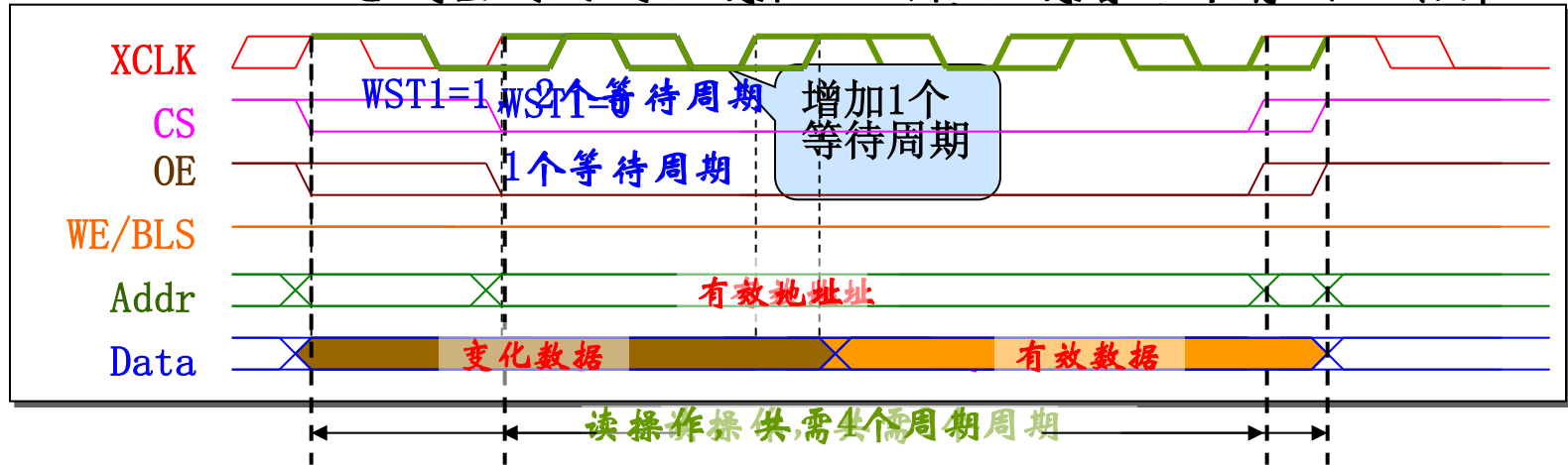
4.6 外部存储器控制器 (EMC)

• 寄存器描述

存储器组配置寄存器0~3(BCFG0~3):

配置寄存器 BCFG0~3	31 : 30	29 : 28	27	26	25	24	23 : 16	15 : 11	10	9 : 5	4	3 : 0
	AT	MW	BM	WP	WPERR	BUSERR	—	WST2	RBLE	WST1	—	IDCY

WST1：外部存储器访问波形(WST1=0)：读操作时，该位控制等待周期。该位控制读访问时存储器内部





4.6 外部存储器控制器（EMC）

• 寄存器描述

存储器组配置寄存器0~3(BCFG0~3):

配置寄存器 BCFG0~3	31 : 30	29 : 28	27	26	25	24	23 : 16	15 : 11	10	9 : 5	4	3 : 0
	AT	MW	BM	WP	WPERR	BUSERR	—	WST2	RBLE	WST1	—	IDCY

RBLE: 字节定位使能信号。在P2290微处理器中，EMC提供了一组字节定位选择信号(BLS0~BLS3)实现对16位或32位外部存储器组的操作。

设定

• 写访

• 读访

操作方式	RBLE	BLS	OE	WE
读操作	0	无效	有效	无效
	1	有效	有效	无效
写操作	0	有效	无效	无效
	1	有效	无效	有效

有效);

有效)。

4.6 外部存储器控制器 (EMC)

• LPC2200 外部总线器件



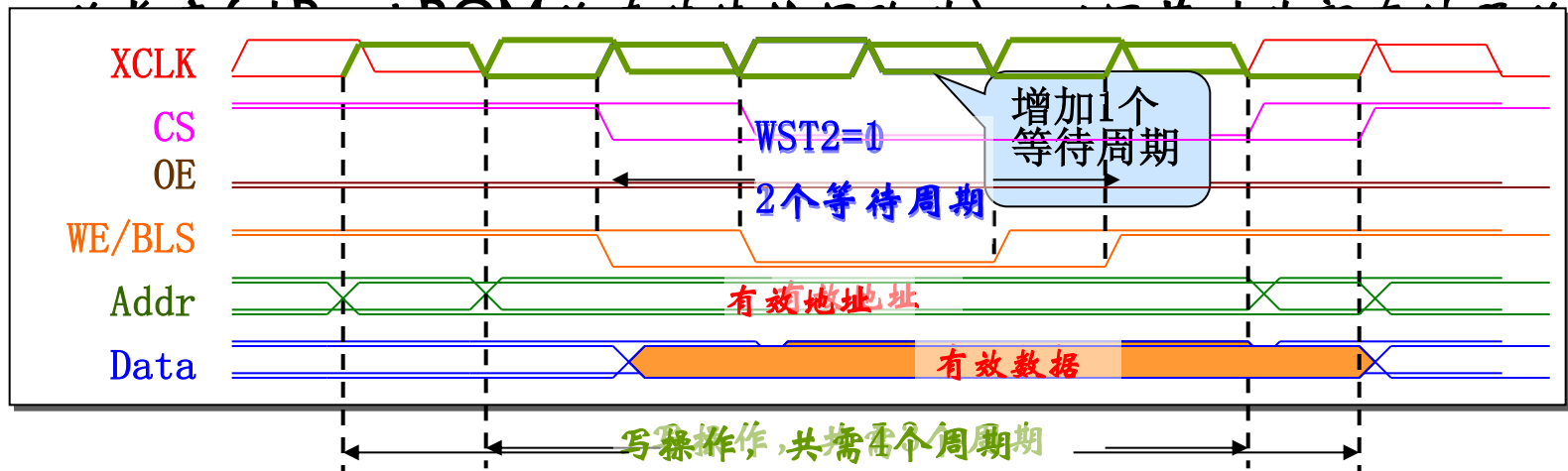
4.6 外部存储器控制器（EMC）

寄存器描述

存储器组配置寄存器0~3(BCFG0~3):

配置寄存器 BCFG0~3	31 : 30	29 : 28	27	26	25	24	23 : 16	15 : 11	10	9 : 5	4	3 : 0
	AT	MW	BM	WP	WPERR	BUSERR	—	WST2	RBLE	WST1	—	IDCY

WST2: 外部存储器写访问长度控制位。和读操作类似，该位控制写访问长度。当WST2=0时，写访问长度为2个等待周期；当WST2=1时，写访问长度为3个等待周期。



4.6 外部存储器控制器（EMC）

• 寄存器描述

存储器组配置寄存器0~3(BCFG0~3):

配置寄存器 BCFG0~3	31 : 30	29 : 28	27	26	25	24	23 : 16	15 : 11	10	9 : 5	4	3 : 0
	AT	MW	BM	WP	WPERR	BUSERR	—	WST2	RBLE	WST1	—	IDCY

MW: 该位控制相应存储器组的数据总线宽度。

BCFGx[29:28]	数据总线宽度
00	8
01	16
10	32
11	保留



4.6 外部存储器控制器（EMC）

• 寄存器描述

MW控制域(BCFG[29:28])与引脚BOOT1:0的关系:

由于Bank0可用于引导程序运行, 所以BCFG0[29:28]的复位值与引脚BOOT1:0的设置有关(当BOOT1:0=11时, 复位后从片内Flash引导程序运行)。

Bank	复位时引脚BOOT1:0的状态	BCFG[29:28]复位值	存储器宽度
0	LL	00	8
0	LH	01	16
0	HL	10	32
0	HH	10	32
1	XX	10	32
2	XX	01	16
3	XX	00	8



4.6 外部存储器控制器（EMC）

- 外部存储器接口

LPC2200的外部存储器宽度可设定为**8位、16位和32位模式**，每一种模式对应的硬件连接都是不同的，原因就是总线工作模式上存在差别。

4.6 外部存储器控制器（EMC）

• 8位总线宽度

数据总线D7~D0有效，字节定位信号中只有BLS0是有效的，总线的工作模式如表。

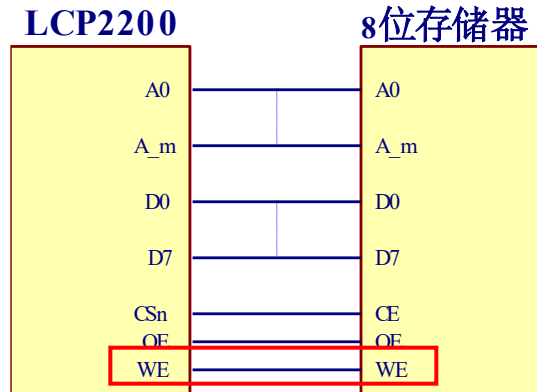
LPC2200工作模式	RBLE	CS	OE	WE	BLS0	说明
读操作 (8位数据)	0	L	L	H	H	——
	1	L	L	H	L	1
写操作 (8位操作)	0	L	H	H	L	2
	1	L	H	L	L	——
空闲操作	X	H	H	H	H	——

说明：

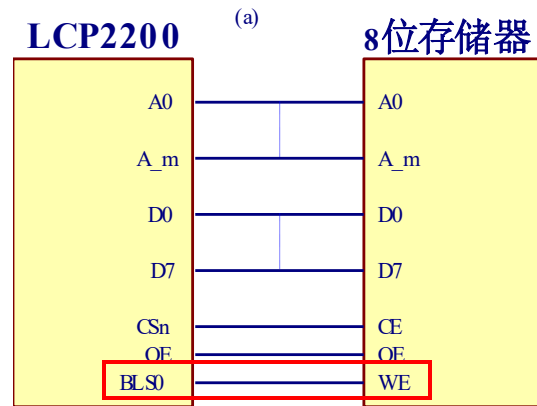
2、此器件每做用BLS0作为片选信号，片选信号能信号，处于低电平状态，取数据不能用作片选信号，片选信号就会同时有效。

4.6 外部存储器控制器（EMC）

• 8位总线宽度连接方式



需要设置 $RBLE = 1$ ，否则在写数据时，WE 不会出现有效信号。



需要设置 $RBLE = 0$ ，否则在读取数据时，读、写使能信号都有效，就会出现总线数据错误。

(b)

4.6 外部存储器控制器（EMC）

• 16位总线宽度

数据总线D15~D0有效，地址总线A0无效，字节定位信号中BLS0和BLS1有效。

随线存储模式表高、低字节定位表示方法：

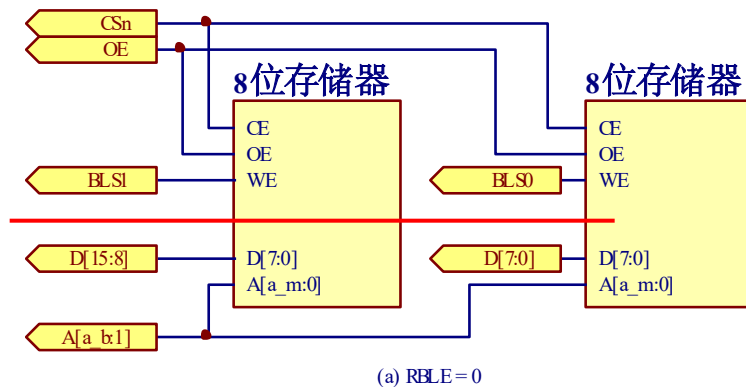
LPC2200工作模式		RBLE	CS	OE	WE	BLS1	BLS0
读操作	8位、16位数据	0	L	L	H	H	H
		1	L	L	H	L	L
写操作	8位数据	0	L	H	H	H	L
						L	H
		1	L	H	L	H	L
						L	H
	16位数据	0	L	H	H	L	L
		1	L	H	L	L	L
空闲操作		X	H	H	H	H	H

LP 接到存

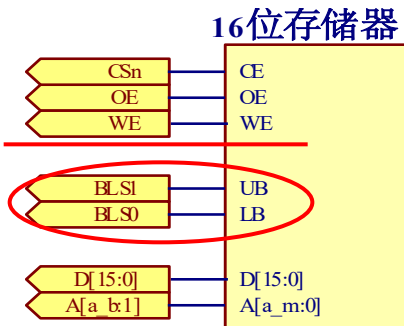
线A1连

4.6 外部存储器控制器（EMC）

• 16位总线宽度连接方式



RBLE = 0。否则在对存储器进行读操作时，读、写使能信号都有效。

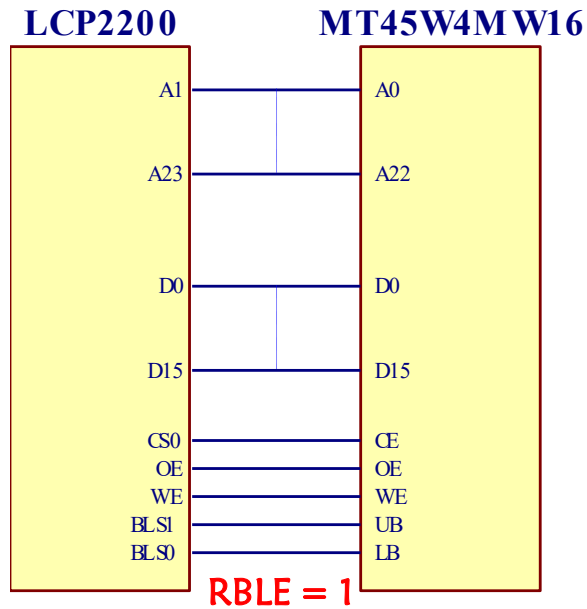


RBLE = 1。否则在对存储器进行写操作时，写使能信号无效。

4.6 外部存储器控制器（EMC）

• 16位总线操作示例

LPC2000与存储芯片MT45W4MW16的连接示意图。MT45W4MW16是一款PSRAM芯片，16位总线宽度，容量8M字节。



MT45W4MW16部分引脚描述	
引脚	描述
A0~A22	地址线
D0~D15	数据线
CE	片选线
OE	读使能线
WE	写使能线
UB	高字节使能线
LB	低字节使能线

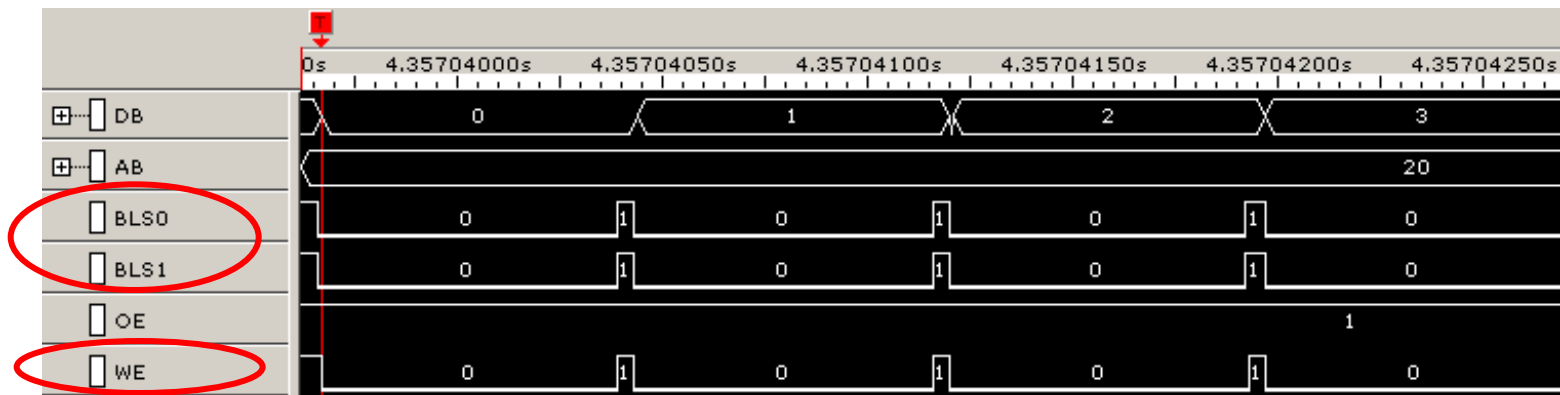
4.6 外部存储器控制器（EMC）

• 向PSRAM中写入16位数据

向PSRAM中0x80001020地址循环写入从0开始递增的数据。

使用逻辑分析仪捕捉到前4次操作的波形。

```
uint16 *point16;
uint16 data16 = 0x00;
point16 = (uint16 *)0x80001020;
while(1)
{
    *point16 = data16++;
}
```



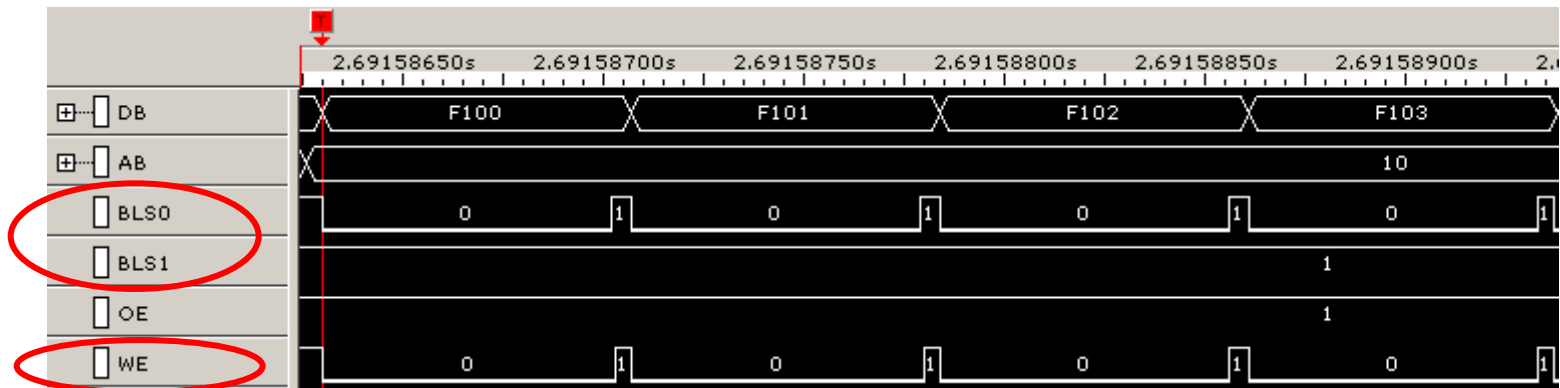
4.6 外部存储器控制器（EMC）

• 向PSRAM中低字节地址写入8位数据

向PSRAM中低字节地址0x80001010循环写入从0开始递增的8位数据。

WE、BLS0有效，数据线上D7~D0有效，D15~D8无效

```
uint8 *point8;
uint8 data8 = 0x00;
point8 = (uint8 *) 0x80001010;
while(1)
{
    *point8 = data8 ++;
}
```



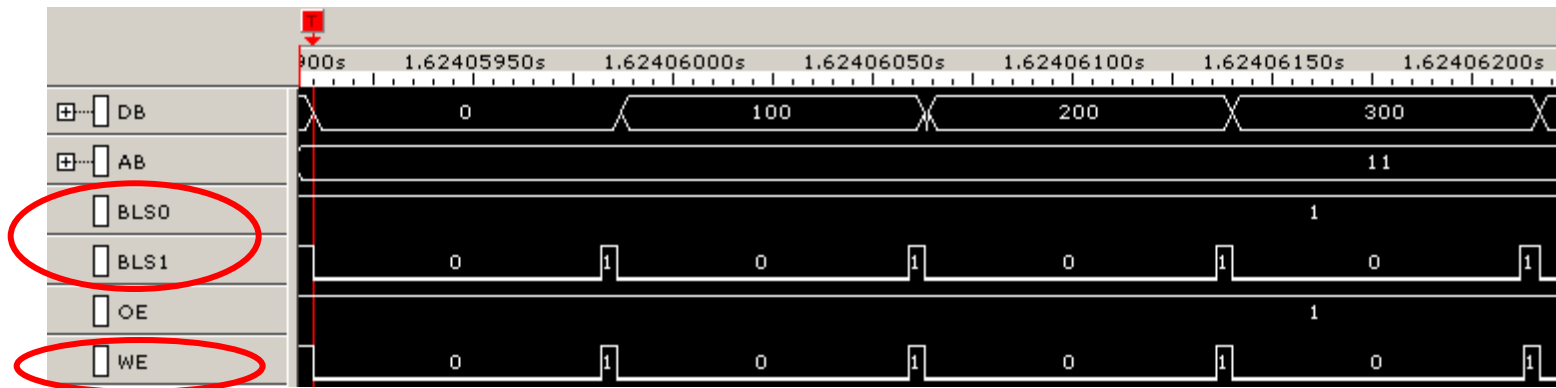
4.6 外部存储器控制器（EMC）

向PSRAM中高字节地址写入8位数据

向PSRAM中高字节地址0x80001011循环写入从0开始递增的8位数据。

WE、BLS1有效，数据线上D15~D8有效，D7~D0无效

```
uint8 *point8;
uint8 data8 = 0x00;
point8 = (uint8 *) 0x80001011;
while(1)
{
    *point8 = data8 ++;
}
```





4.6 外部存储器控制器（EMC）

- 16位写操作总结

1、写入16位数据时，EMC将16位数据输出到D[15:0]，同时使BLS0和BLS1均有效；

2、向“低字节地址”处写入8位数据时，EMC将8位数据输出到D[7:0]，同时使BLS0输出有效，BLS1输出无效；

3、向“高字节地址”处写入8位数据时，EMC将8位数据输出到D[15:8]，同时使BLS1输出有效，BLS0输出无效。

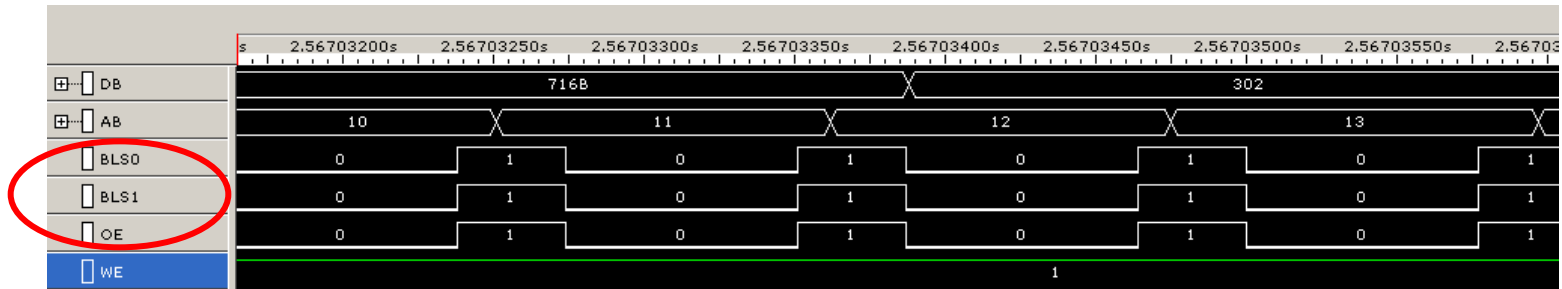
4.6 外部存储器控制器（EMC）

• 从PSRAM中读取8位数据

从PSRAM 0x8000 1010 ~
0x8000 1013 循环读取8位数据

- 读取16位数据时，BLS0和BLS1均有效，有效数据位于：D[15:0]；
- 读取“低字节”数据时，BLS0和BLS1均有效，有效数据位于：D[7:0]；
- 读取“高字节”数据时，BLS0和BLS1均有效，有效数据位于：D[15:8]。

```
uint8 RcvData8;
uint8 *point8;
point8 = (uint8 *) 0x80001010;
while(1)
{
    RcvData8 = * point8++;
    RcvData8 = * point8++;
    RcvData8 = * point8++;
    RcvData8 = * point8--;
    point8--;
    point8--;
}
```



4.6 外部存储器控制器（EMC）

• 32位总线宽度

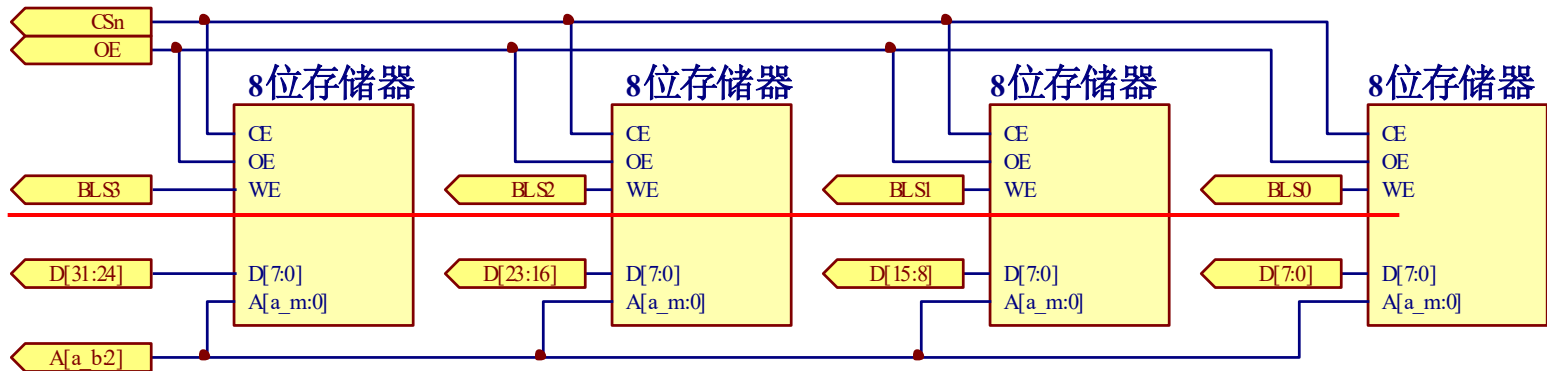
数据总线D31~D0有效，地址总线A1、A0无效，字节定位信号中BLS0~BLS3有效。32位存储器件中字节0、1、2、3的定位表示法：

32位存储器数据				32位存储器件地址
D31 ~ D24	D23 ~ D16	D15 ~ D8	D7 ~ D0	
字节3	字节2	字节1	字节0	0x0000 0000
字节3	字节2	字节1	字节0	0x0000 0001
字节3	字节2	字节1	字节0	0x0000 0010
字节3	字节2	字节1	字节0	

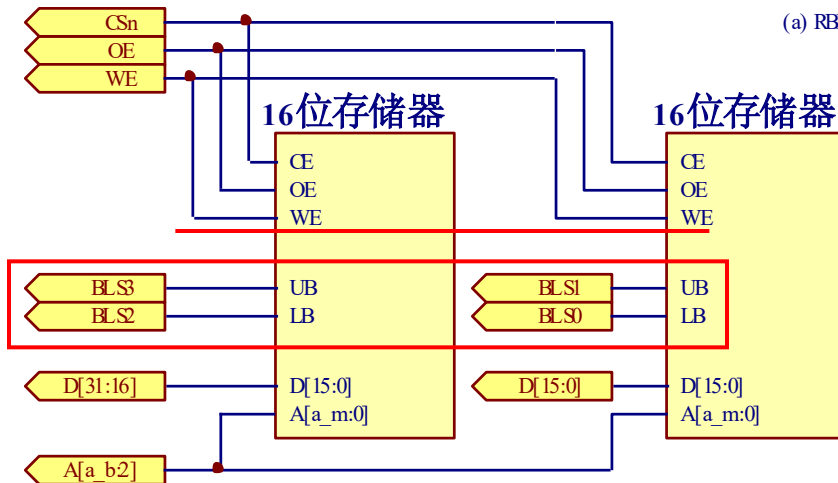
LPC2200操作32位存储器时，需将LPC2200的地址线A2连接到存储器地址线A0处。

4.6 外部存储器控制器（EMC）

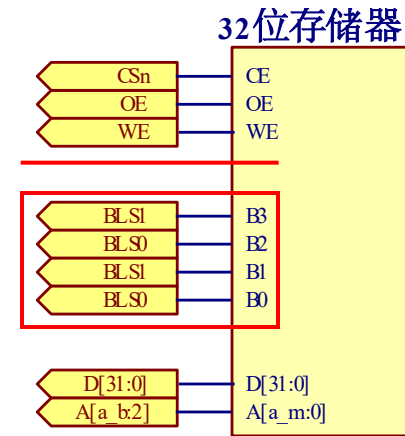
• 32位总线宽度连接方式



(a) RBLE = 0



(b) RBLE = 1



(c) RBLE = 1



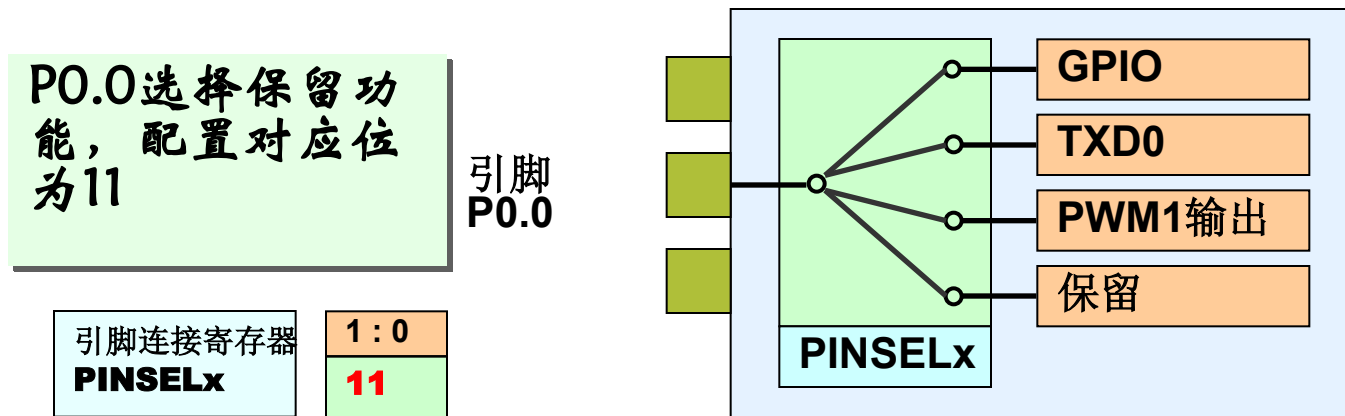
LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.7 引脚连接模块

概述

LPC2000系列微控制器的大部分管脚都具有多种功能，即管脚复用，但是同一引脚在同一时刻只能使用其中一个功能，通过配置相关寄存器控制多路开关来连接引脚与片内外设。



4.7 引脚连接模块

- 寄存器描述

LPC2000系列微控制器具有三个32位宽度PINSEL寄存器，其中PINSEL0和PINSEL1控制端口0，PINSEL2根据芯片的不同控制的端口数量也不同。

寄存器	LPC2100	LPC2200
PINSEL0	P0[0:15]	
PINSEL1	P0[16:31]	
PINSEL2	P1[16:31]	P1[0:1]、 P1[16:31]、 P2[0:31]、 P3[0:31]

4.7 引脚连接模块

• PINSEL2寄存器与外部总线

PINSEL2[5:4]与数据总线和控制线的关系列表

PINSEL2 [5:4]	数据总线宽度	P2. 27:15	P2. 15:8	P2. 7:0	P3. 31	P3. 3 0	P3. 29:28	P1. 1	P1. 0
10	32位	D[31:16]	D[15:0]	D[7:0]	BLS0	BLS1	BLS3:2	OE	CS0
01	16位	PINSEL2 [20;22] 控制	D[15:0]	D[7:0]	BLS0	BLS1	PINSEL2 [6;7] 控制	OE	CS0
00	8位		GPIO	D[7:0]	BLS0	GPIO		OE	CS0
11	无		GPIO	GPIO	GPIO	GPIO		GPIO	GPIO

PINSEL2[27:25]与地址线的关系列表

PINSEL2 [27:25]	000	001	010	011	100	101	110	111
地址线宽度	无地址线	A3:2	A5:2	A7:2	A11:2	A15:2	A19:2	A23:2

4.7 引脚连接模块

- 应用示例

要求将P0.8、P0.9设置为TxD1、RxD1

通过查阅PINSEL0寄存器设置表，得到P0.9和P0.8的控制位为PINSEL0[19:16]，当该域设置为[0101] (0x05)时选择 RxD1和 TxD1。

```
PINSEL0 = 0x05 << 16;
```

为了不影响别的管脚连接设置，通常选择下面的设置方法。

```
PINSEL0 = (PINSEL0 & 0xFFFF0FFF) | (0x05 << 16);
```



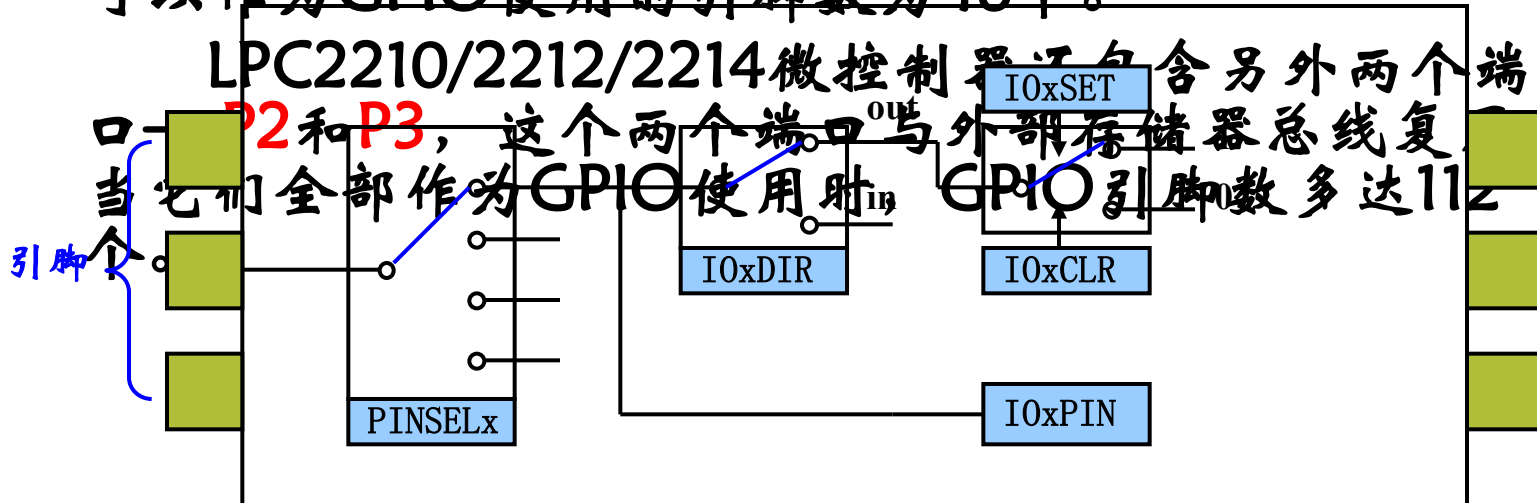
LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

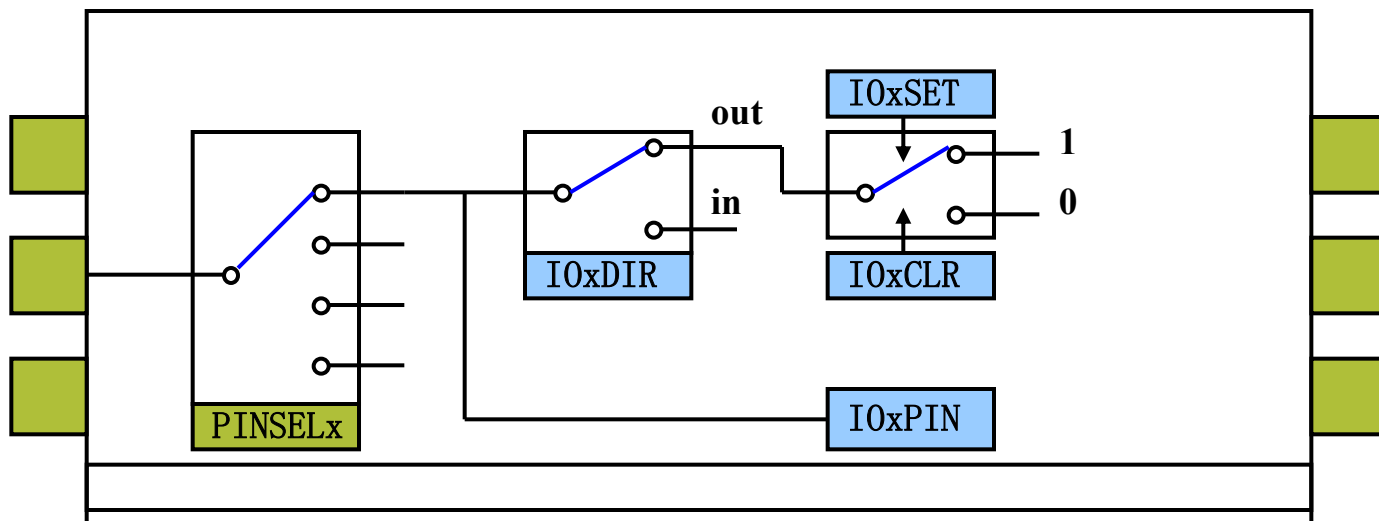
引脚描述

LPC2114/2134微控制器具有两个端口——P0和P1，
可以作为GPIO使用的引脚数为46个。

LPC2210/2212/2214微控制器还包含另外两个端口 **P2** 和 **P3**，这个两个端口与外部存储器总线复用时，GPIO引脚数多达112。

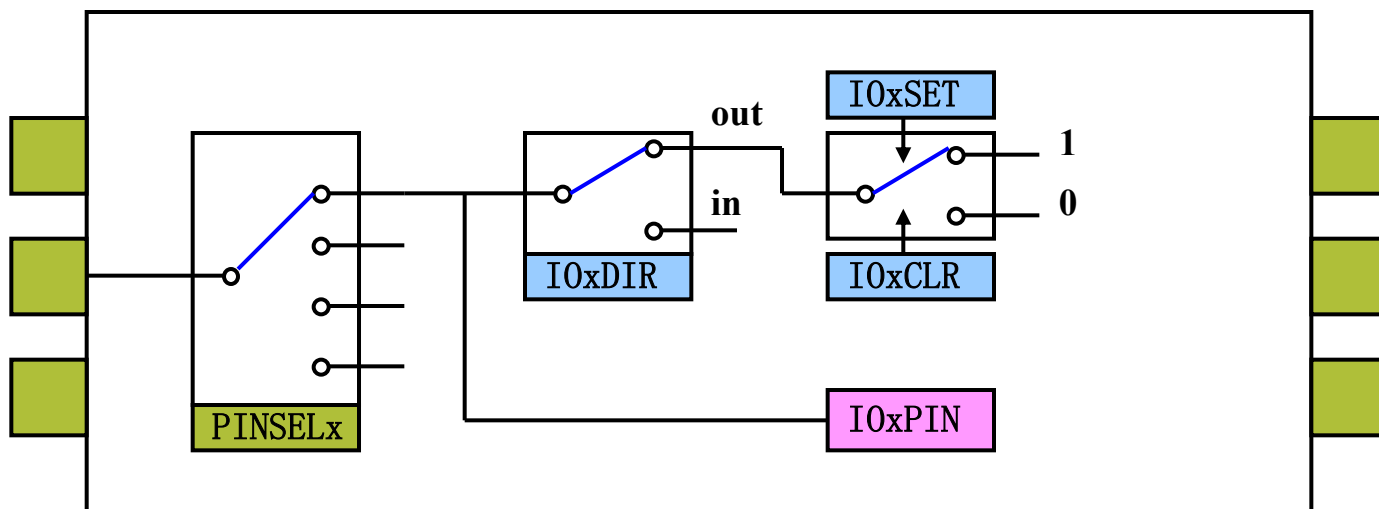


GPIO相关寄存器描述



通用名称	描述	访问类型	复位值
IOPIN	GPIO引脚值寄存器，不管方向模式如何，引脚的当前状态都可以从该寄存器中读出	只读	NA
IOSET	GPIO输出置位寄存器。该寄存器控制引脚输出高电平	读/置位	0x00000000
IOCLR	GPIO输出置位寄存器。该寄存器控制引脚输出低电平	只清零	0x00000000
IODIR	GPIO方向控制寄存器。该寄存器单独控制每个IO口的方向	读/写	0x00000000

■ GPIO相关寄存器描述——IOxPIN



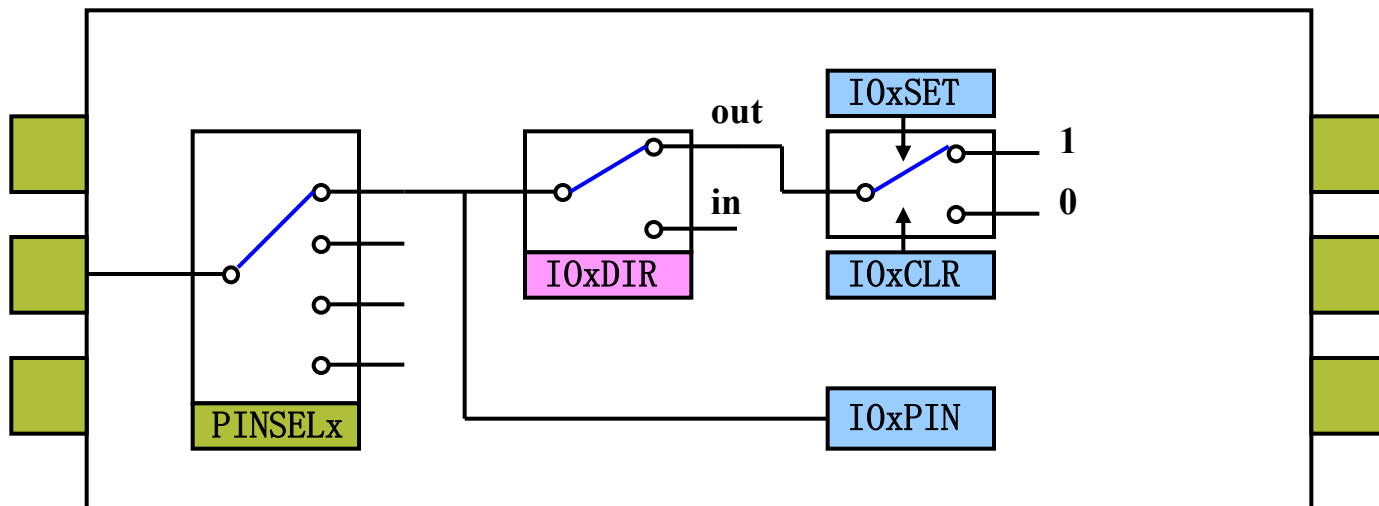
IOxPIN	描述	复位值
31 : 0	GPIO引脚值。IOxPIN[0]对应于Px.0 ... IOxPIN[31]对应于Px.31引脚	未定义

该寄存器反映了当前引脚的状态。IOxPIN中的x对应于某一个端口，如P1口对应于IO1PIN。所以芯片存在多少个端口，就有多少个IOxPIN分别与之对应。

写该寄存器会将值保存到输出寄存器，具体使用稍后介绍。

注意：无论引脚被设置为输入还是输出模式，都不影响引脚状态的读出。

■ GPIO相关寄存器描述——IOxDIR

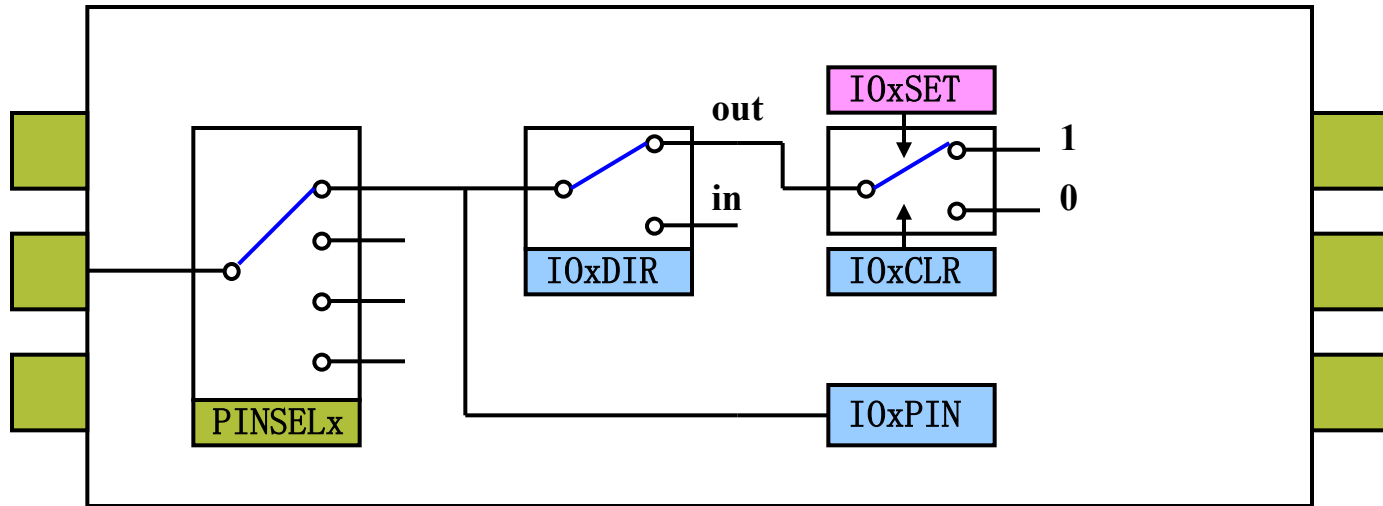


IOxDIR	描述	复位值
31 : 0	方向控制位。IOxDIR[0]对应于Px.0 ... IOxDIR[31]对应于Px.31引脚	0

当引脚设置为GPIO输出模式时，可使用该寄存器控制引脚的方向。
向某位写入1使对应引脚作为输出功能，写入0时作为输入功能。

作为输入功能时，引脚处于高阻态。

■ GPIO相关寄存器描述——IOxSET

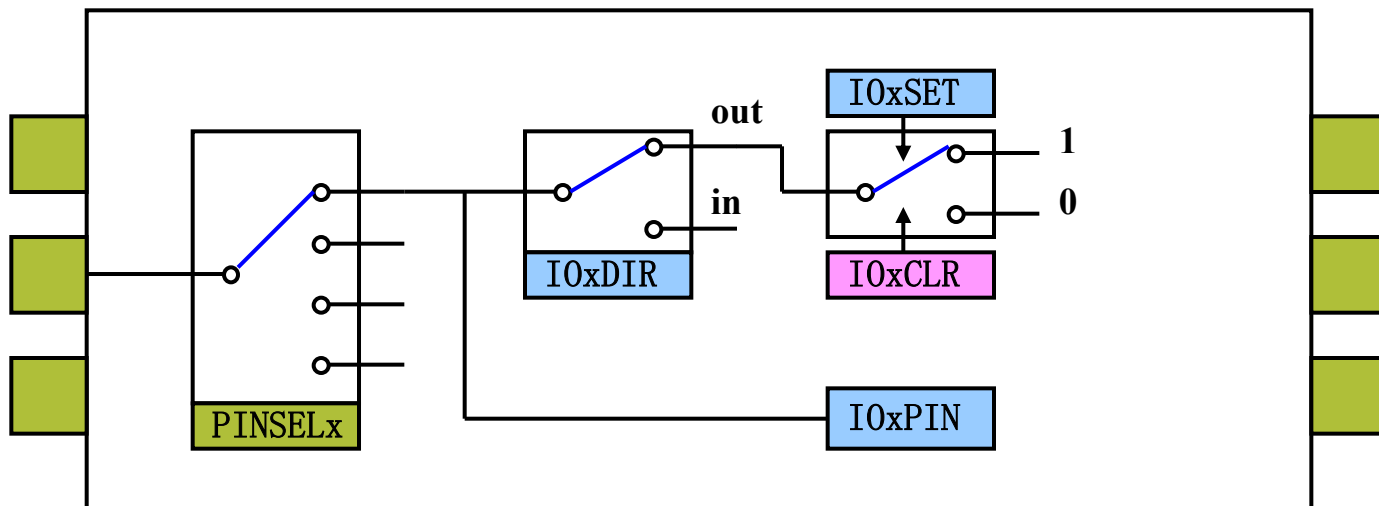


IOxSET	描述	复位值
31 : 0	输出置位。IOxSET[0]对应于Px.0 ... IOxPIN[31]对应于Px.31引脚	0

当引脚设置为GPIO输出模式时，可使用该寄存器从引脚输出高电平。向某位写入1使对应引脚输出高电平。写入0无效。

从该寄存器读回的数据为GPIO输出寄存器的值。该值不反映外部环境对引脚的影响。

■ GPIO相关寄存器描述——IOxCLR



IOxCLR	描述	复位值
31 : 0	输出清零。IOxCLR[0]对应于Px.0 ... IOxCLR[31]对应于Px.31引脚	0

当引脚设置为GPIO输出模式时，可使用该寄存器从引脚输出**低**电平。
向某位写入1使对应引脚输出低电平。写入0无效。

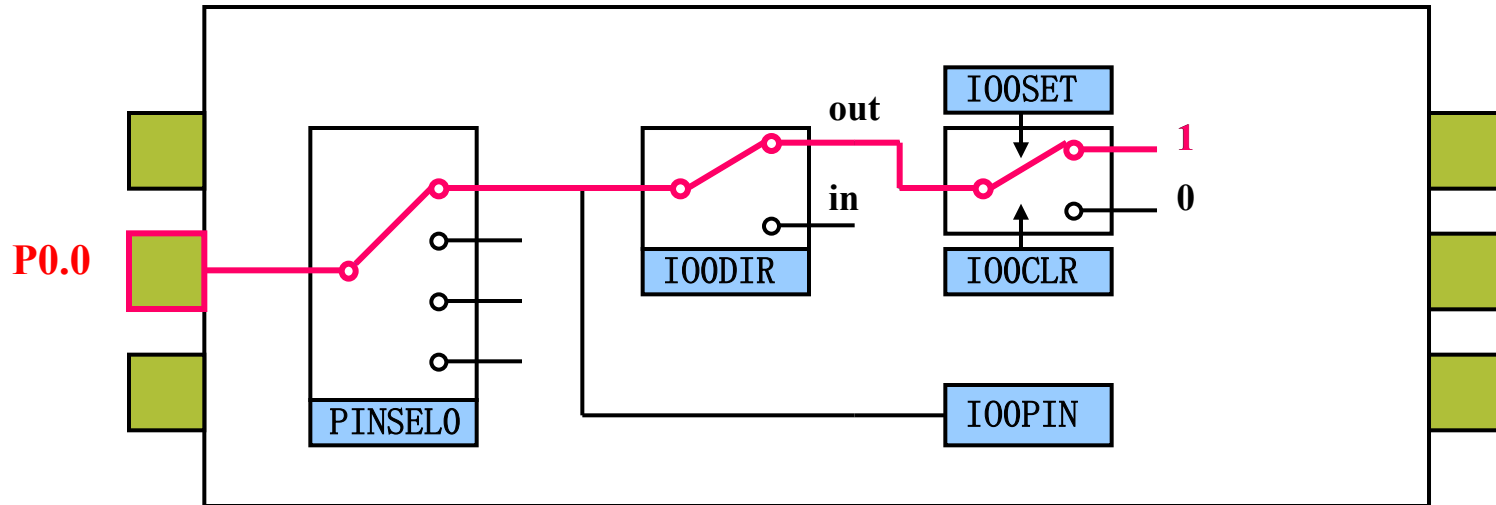
注意：读取该寄存器无效，不能读回输出寄存器的值。

4.8 GPIO

■ 使用GPIO注意要点

- 引脚设置为输出方式时，输出状态由IOxSET和IOxCLR中最后操作的寄存器决定；
- 大部分GPIO输出为**推挽**方式（个别引脚为开漏输出），正常拉出/灌入电流均为4mA（短时间极限值40mA）；
- 复位后默认所有GPIO为输入模式。

GPIO应用示例——设置P0.0输出高电平



C代码:

• • •

```
PINSEL0 &= 0xFFFFF0FC;
```

```
IOODIR    = 0x00000001;
```

~~IOOSET = 0x00000001;~~

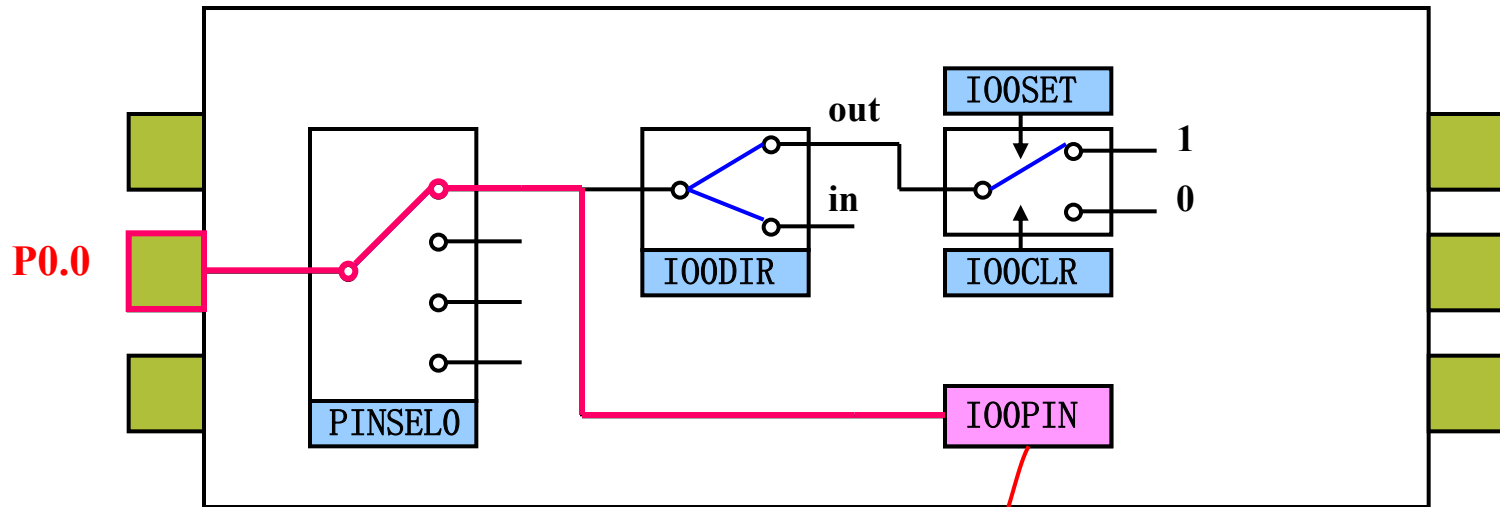
• • •

(1) 设置引脚连接模块, P0.0为GPIO

(2) 设置P0.0口方向，设置为输出

(3) 设置P0.0口状态，输出高电平

GPIO应用示例——读取P0.0引脚状态



C代码:

```
...  
uint32 PinStat;
```

```
PINSEL0 &= 0xFFFFF0FC;
```

```
I00DIR &= 0xFFFFF0FE;
```

```
PinStat = I00PIN;
```

```
...
```

(1) 设置引脚连接模块，P0.0为GPIO

(2) 设置P0.0口方向，设置为输入

(3) 从IO0PIN读取引脚状态



GPIO应用示例——输出多位数据至IO口

在需要将多位数据同时输出到某几个IO口线时，通常使用IOxSET和IOxCLR来实现，在某些情况下也可以使用IOxPIN寄存器实现。后者可以在多个IO口上直接输出0和1电平。

本例将8位无符号整型变量Data的值输出到P0.0~P0.7。

使用IOxSET和IOxCLR实现：

```
#define DataBus 0xFF
```

```
PINSEL0 &= 0xFFFF0000;
```

(1) 设置引脚连接模块，P0.0~7为GPIO

```
I00DIR |= DataBus;
```

(2) 设置P0.0口方向，设置为输出

```
I00CLR = DataBus;
```

(3) 清零8位IO口的输出状态

```
I00SET = Data;
```

(4) Data变量中为1的位将输出高电平

...

数据输出线：

0x??

0x00

Data



GPIO应用示例——输出多位数据至IO口

在需要将多位数据同时输出到某几个IO口线时，通常使用IOxSET和IOxCLR来实现，在某些情况下也可以使用IOxPIN寄存器实现。后者可以在多个IO口上直接输出0和1电平。

本例将8位无符号整数变量Data的值输出到P0.0~P0.7。

使用IOxPIN实现：

```
#define DataBus 0xFF
```

```
PINSEL0 &= 0xFFFF0000;
```

(1) 设置引脚连接模块，P0.0为GPIO

```
IO0DIR |= DataBus;
```

(2) 设置P0.0口方向，设置为输出

```
IO0PIN = (IO0SET & 0xFFFFFFF0) | Data;
```

(3) 写IO0PIN，输出数据

...

数据输出线：

0x??

Data

与前者对比：

0x??

0x00

Data



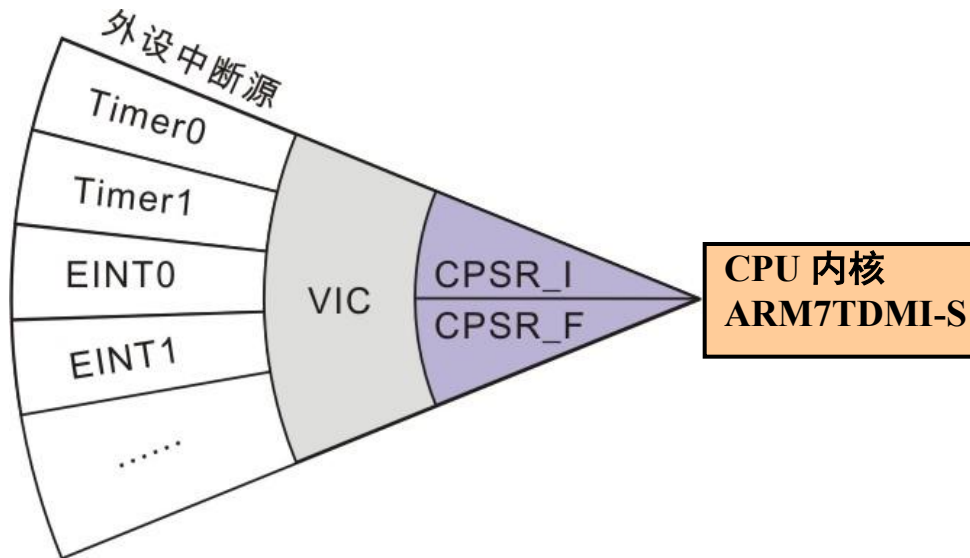
LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块(MAM)
- 6.外部存储器控制器(EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.9 向量中断控制器

- 概述

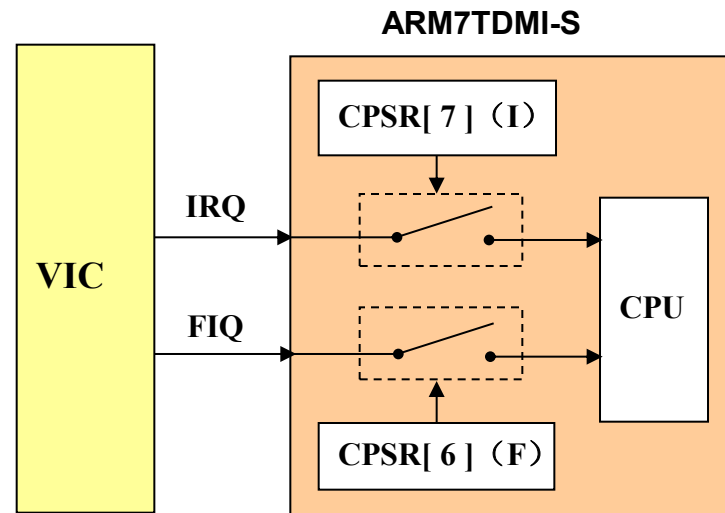
ARM7TDMI内核具有两个中断输入，分别为IRQ中断和FIQ中断。向量中断控制器 (VIC) 负责管理芯片的中断源，最多可以管理32个中断输入请求。



4.9 向量中断控制器

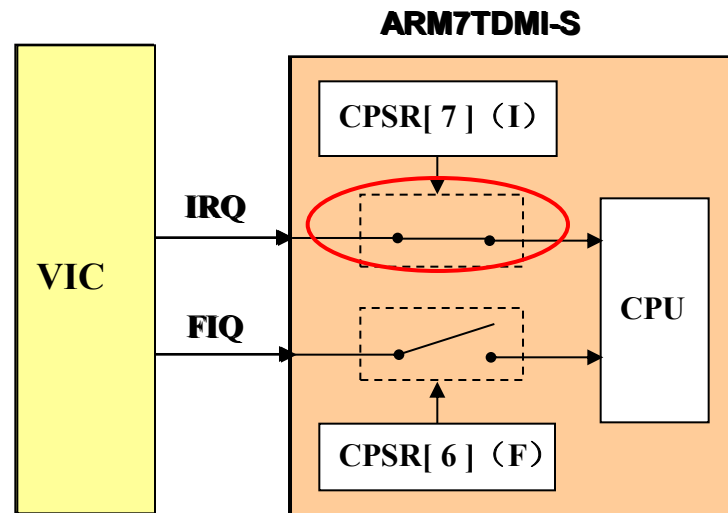
- 程序状态寄存器CPSR与VIC的关系

ARM内核通过CPSR来监视和控制内部的操作，CPSR中的“I”位和“F”位分别用来控制IRQ模式和FIQ模式的使能。



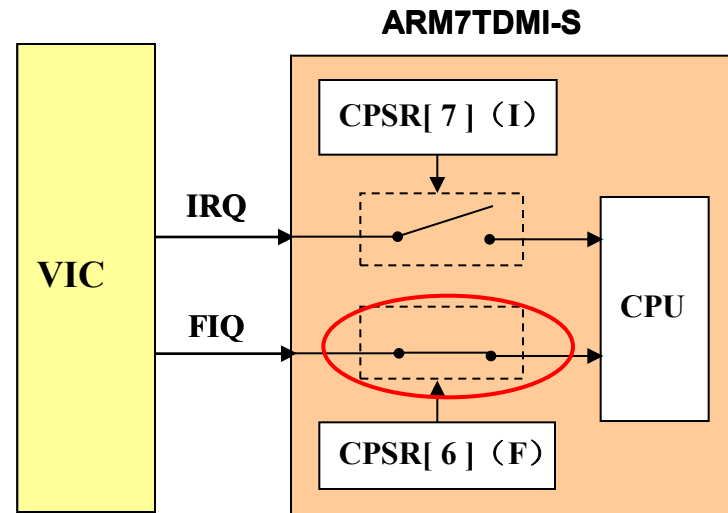
4.9 向量中断控制器

- 程序状态寄存器CPSR与VIC的关系
 - 当 $I = 1$ 时，禁止IRQ中断
 - 当 $I = 0$ 时，使能IRQ中断



4.9 向量中断控制器

- 程序状态寄存器CPSR与VIC的关系
 - 当 $F = 1$ 时，禁止FIQ中断
 - 当 $F = 0$ 时，使能FIQ中断

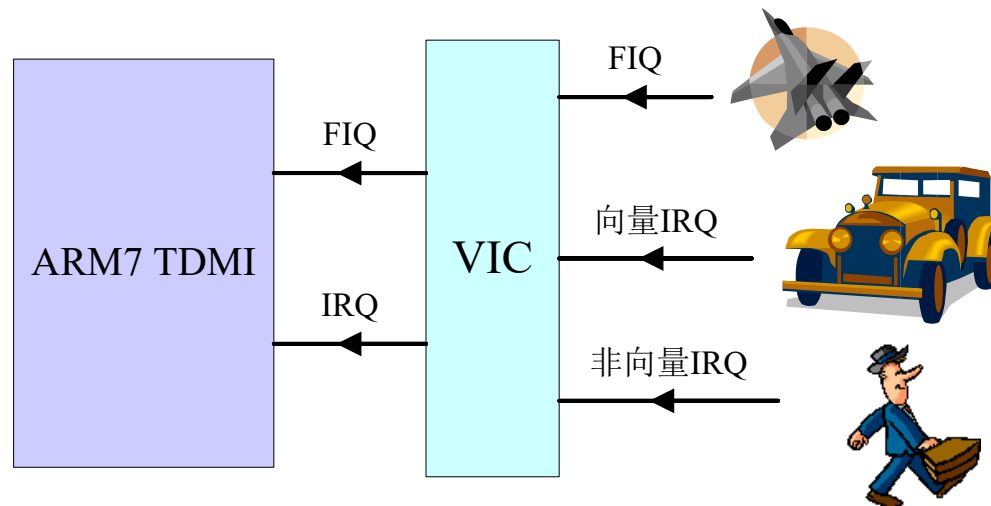


4.9 向量中断控制器

中断分类

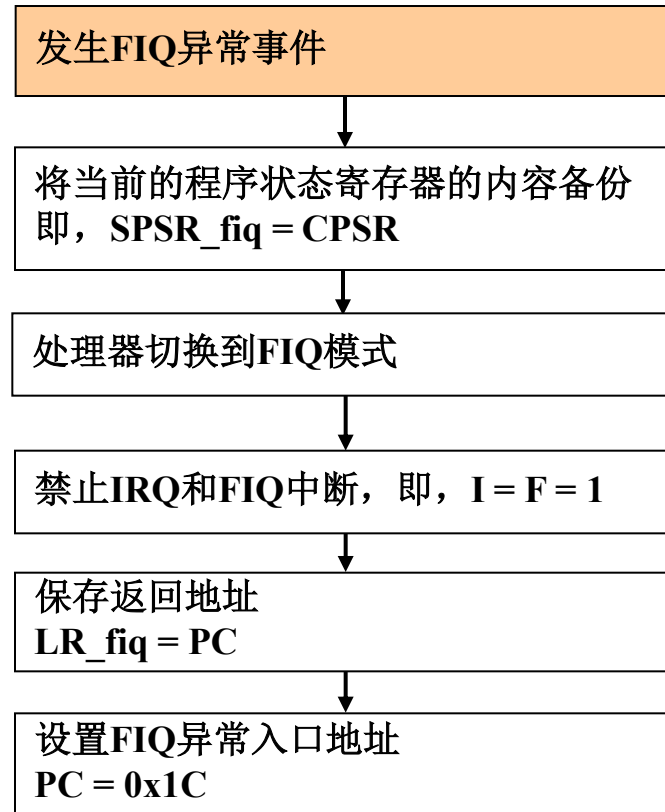
中断输入请求可以在VIC中被设置为以下三类：

- **FIQ中断**：具有最高优先级；
- **向量IRQ中断**：具有中等优先级；
- **非向量IRQ中断**：具有最低优先级；



4.9 向量中断控制器

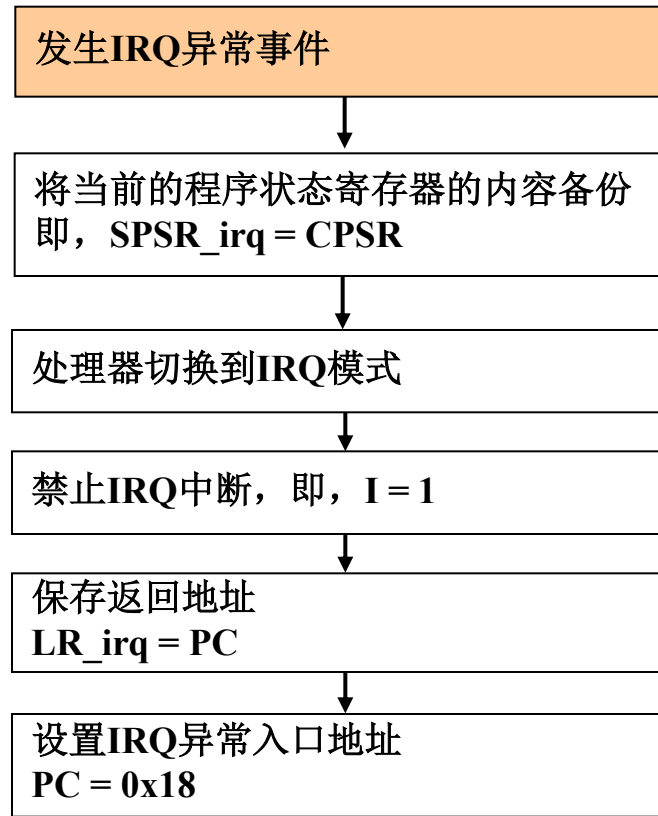
• FIQ中断硬件处理流程



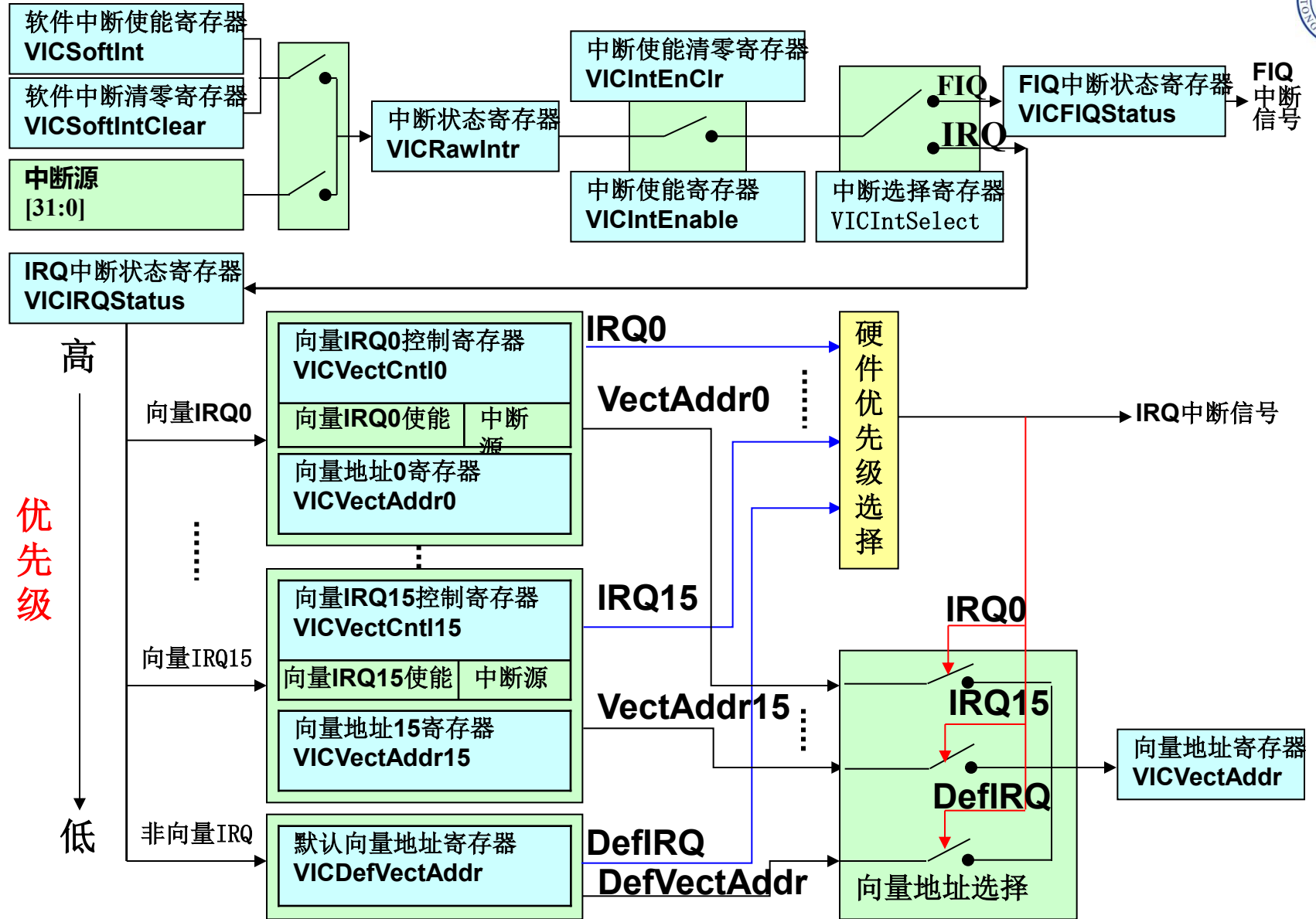
说明: ARM7不支持FIQ中断嵌套

4.9 向量中断控制器

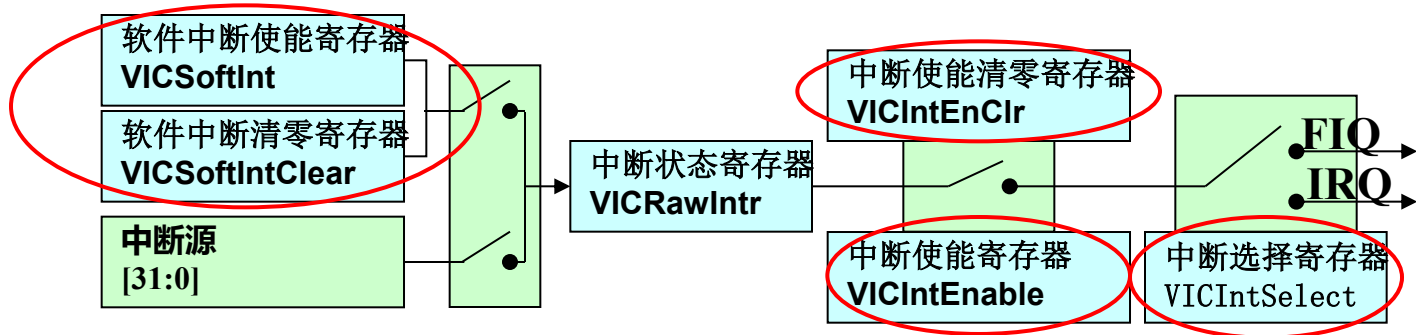
• IRQ中断相应流程



说明: ARM7不支持IRQ中断嵌套



寄存器描述—控制寄存器

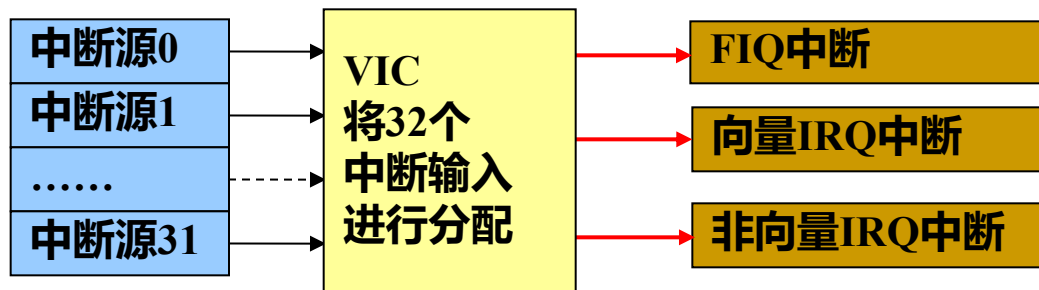


名称	描述	访问	复位值	地址
VICIntEnable	中断使能寄存器 控制32个中断请求（包括软件中断）的使能	R/W	0	0xFFFF F010
VICIntEnClr	中断使能清零寄存器 将中断使能寄存器中的一个或多个位清零	W	0	0xFFFF F014
VICIntSelect	中断选择寄存器 将32个中断请求的每个中断分配为FIQ或IRQ	R/W	0	0xFFFF F00C
VICSoftInt	软件中断寄存器 控制对应通道产生软件中断	R/W	0	0xFFFF F018
VICSoftIntClear	软件中断清零寄存器 禁止对应通道的软件中断	W	0	0xFFFF F01C

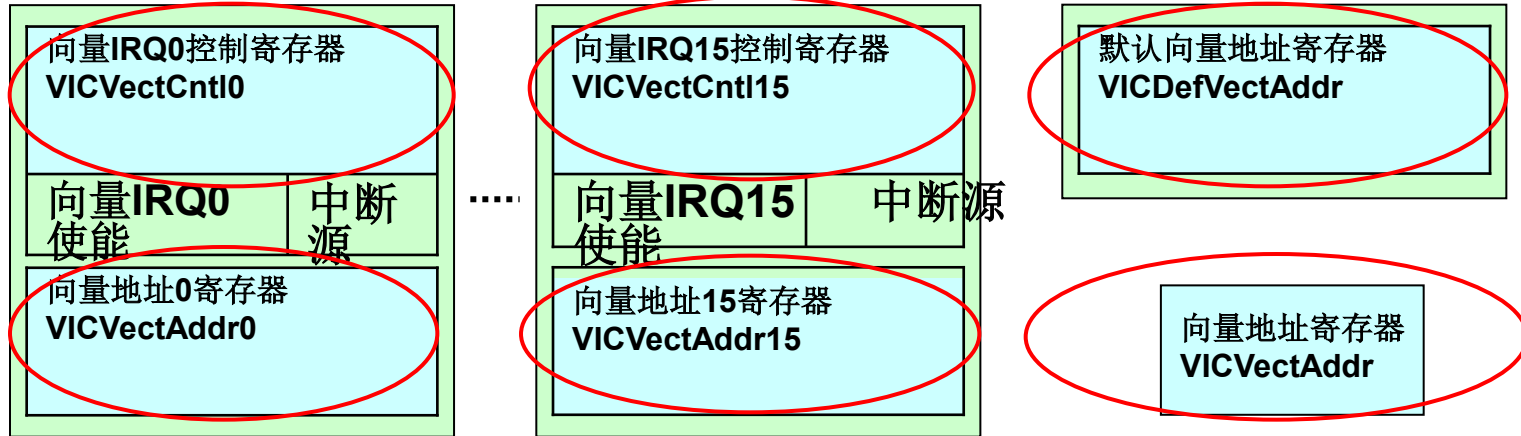
4.9 向量中断控制器

- 控制寄存器功能描述

控制寄存器	功能描述
VICIntEnable	使能（禁止）中断源产生中断
VICIntEnClr	
VICIntSelect	中断类型选择：FIQ & IRQ
VICSoftInt	软件中断设置寄存器
VICSoftIntClear	



寄存器描述—参数设置寄存器



名称	描述	复位置	地址
VICVectCntl0	向量控制0寄存器	0	0xFFFF F200
~	~		~
VICVectCntl15	向量控制15寄存器		0xFFFF F23C
VICVectAddr0	向量地址0寄存器	0	0xFFFF F100
~	~		~
VICVectAddr15	向量地址15寄存器		0xFFFF F13C
VICDefVectAddr	默认向量地址寄存器	0	0xFFFF F034
VICVectAddr	向量地址寄存器	0	0xFFFF F030



4.9 向量中断控制器

• IRQ中断特点

	特点	区别
向量 IRQ	● 支持16个向量IRQ中断; ● 16个优先级; ● 每个优先级指定一个服务程序入口地址。	能为每个非向量IRQ中断源设置服务程序地址。
非向量 IRQ	● 支持1个非向量IRQ中断; ● 所有非向量IRQ的入口地址都相同。	所有的非向量IRQ中断都共用一个相同的服务程序入口地址。

4.9 向量中断控制器

• IRQ中断相关寄存器

寄存器名称	功能
VICVectCnt10~15	为中断源分配向量IRQ中断的优先级。
VICVectAddr0~15	为该中断优先级设置服务程序入口地址。
VICDefVectAddr	设置非向量中断服务程序的入口地址。

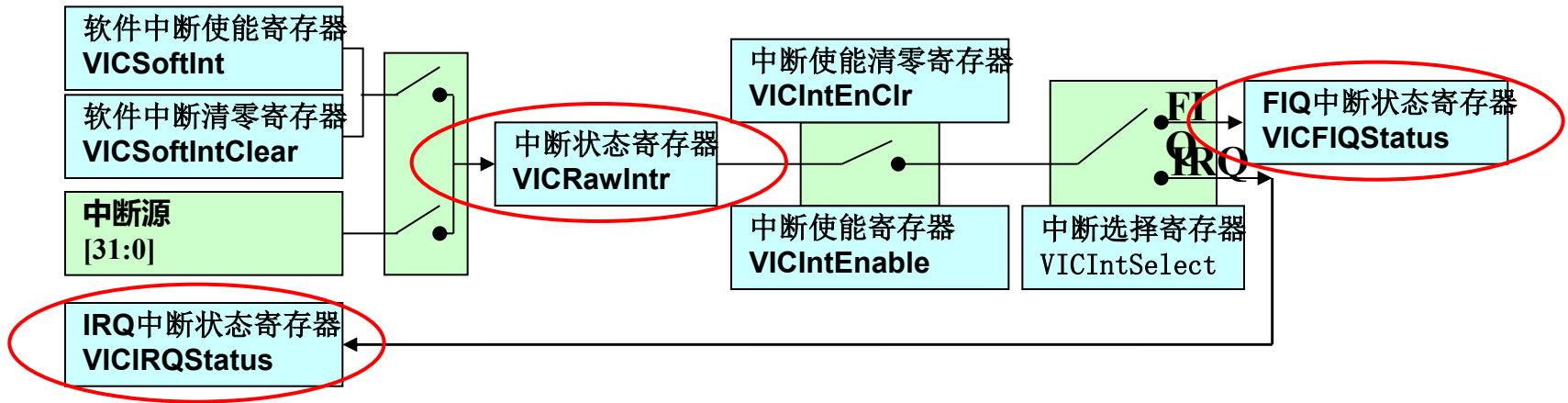
注意：如果将同一个中断源分配给多个使能的向量IRQ中断，那么该中断源发生中断时，会使用最高优先级（最低编号）的寄存器设置。

4.9 向量中断控制器

- 产生中断后的服务程序地址

向量IRQ中断	发生向量IRQ中断后，VIC将对应的向量地址寄存器中的数据存入VICVectAddr寄存器中。
非向量IRQ中断	发生非向量IRQ中断后，VIC将默认向量地址寄存器中的数据存入VICVectAddr寄存器中

寄存器描述—状态寄存器



名称	描述	访问	复位值	地址
VICIRQStatus	IRQ状态寄存器 该寄存器读出定义为IRQ并使能的中断的状态	RO	0	0xFFFF F000
VICFIQStatus	FIQ状态请求 该寄存器读出定义为FIQ并使能的中断的状态	RO	0	0xFFFF F004
VICRawIntr	所有中断的状态寄存器 该寄存器 读出32个中断请求/软件中断的状态，不管中 断是否使能或分类	RO	0	0xFFFF F008

注意：读取VICRawIntr寄存器将得到所有32个中断请求和软件中断的状态，它不管中断是否使能或分类。

4.9 向量中断控制器

- 保护使能寄存器

在某些场合可能需要禁止在用户模式下访问VIC寄存器，以提高软件的安全等级。

保护使能寄存器 (VICProtection)：

位	[31:1]	0
功能	—	当该位为1时，只能在特权模式下访问VIC寄存器。

4.9 向量中断控制器

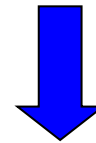
• FIQ中断处理

硬件处理

①	SPSR_fiq = CPSR
②	CPSR = nzcqvq I Ft_fiq
③	LR_fiq = PC
④	PC = 0x1C

软件处理

①	中断服务程序
②	设置返回地址
③	恢复程序状态寄存器CPSR



SUBS PC, LR, #4

4.9 向量中断控制器

• IRQ中断处理

硬件处理

①	SPSR_irq = CPSR
②	CPSR = nzcvc q ift_irq
③	VICVectAddr = VICVectAddrn
④	LR_irq = PC
⑤	PC = 0x18

软件处理

①	获取中断服务程序地址
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR

4.9 向量中断控制器

• IRQ中断处理

硬件处理

①	SPSR_irq = CPSR
②	CPSR = nzcvc q ift_irq
③	VICVectAddr = VICVectAddrn
④	LR_irq = PC
⑤	PC = 0x18

软件处理

①	PC ← [VICVectAddr]
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR

4.9 向量中断控制器

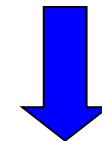
• IRQ中断处理

硬件处理

①	SPSR_irq = CPSR
②	CPSR = nzcvc I ft_irq
③	VICVectAddr = VICVectAddrn
④	LR_irq = PC
⑤	PC = 0x18

软件处理

①	LDR PC, [PC, #-0xff0]
②	执行中断服务程序
③	设置返回地址
④	恢复程序状态寄存器CPSR



SUBS PC, LR, #4

• IRQ中断的设计实例

将定时器0中断分配为向量IRQ通道0，中断服务程序地址设置为Timer0_ISR。

步骤：

中断选择寄存器
VICIntSelect

中断通道4分配为IRQ中断
(定时器0位于中断通道#4)

C代码：

```
VICIntSelect = 0x00000000;
```

IRQ通道0向量地址寄存器
VICVectAddr0

定时器0中断服务程序地址
Timer0_ISR

```
VICVectAddr0 = (int)Timer0_ISR;
```

向量IRQ通道0控制寄存器位分配

IRQ通道0控制寄存器
VICVectCntl0

31: 6	5	4: 0	位
0	1	0x04	数值

```
VICVectCntl0 = (0x20 | 4);
```

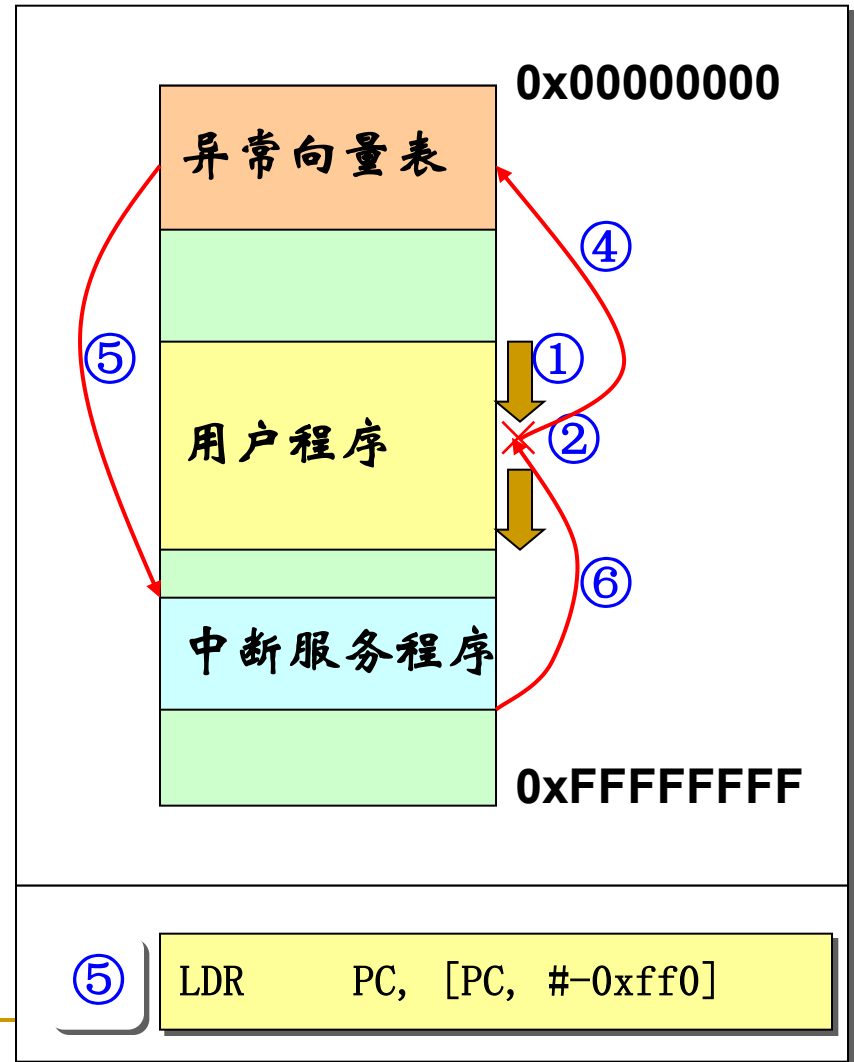
中断使能寄存器
VICIntEnable

中断通道#4对应位，置“1”

```
VICIntEnable = (1 << 4);
```

图示IRQ中断的发生过程

- 1.正在执行用户程序；
- 2.外部中断0发生中断；
- 3.VIC硬件将中断服务程序地址装入VICVectAddr寄存器；
- 4.程序跳转至异常向量表中IRQ入口0x0018处；
- 5.执行指令跳转至VICVectAddr寄存器中的中断服务地址；
- 6.中断服务程序执行完毕，返回被中断的用户程序继续执行被中断的代码。



图示IRQ中断的发生过程

注意：

任何IRQ中断产生之后，微控制器跳转到地址0x18执行代码，该地址处的指令为：

LDR PC, [PC, #-0xff0]

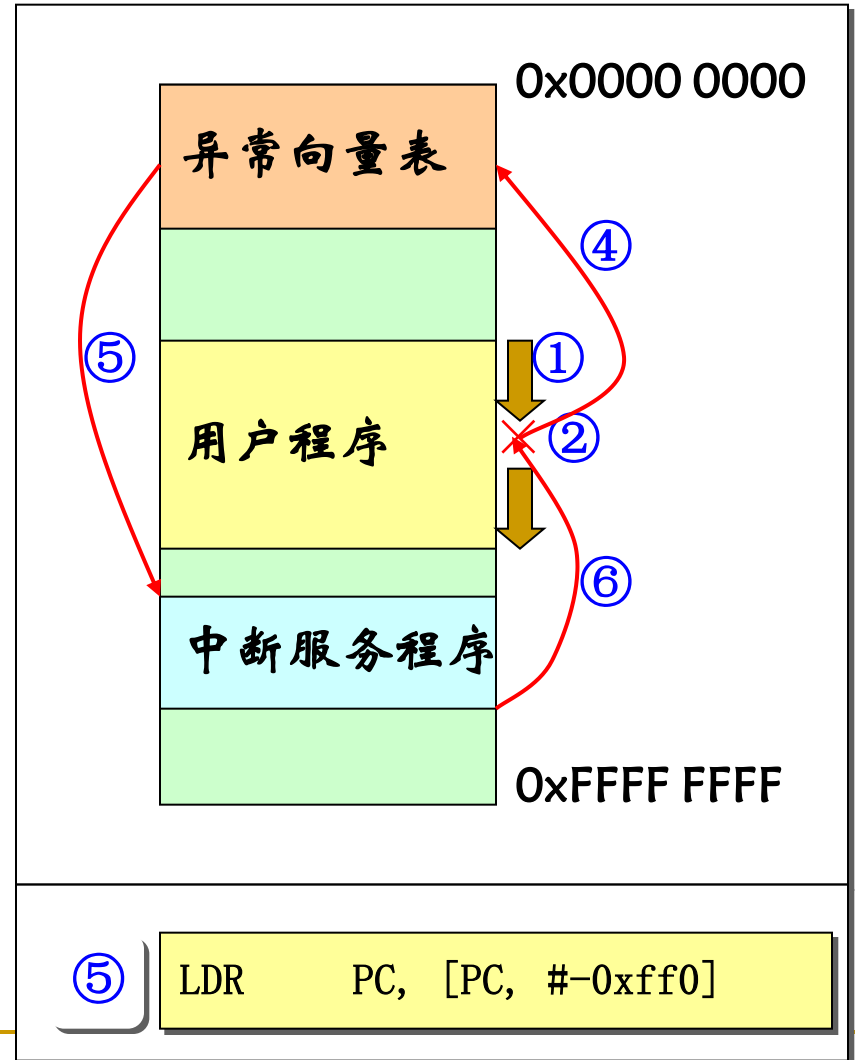
在执行该指令时，PC指向当前指令后的第2条指令，即地址0x20。

该指令将内存地址：

0x20-0xFF0 = 0xFFFFF030

处的内容存入PC，该地址正是VICVectAddr寄存器的地址。

所以一条ARM指令即可实现程序向中断服务程序的跳转。



4.9 向量中断控制器

• 使用VIC的注意事项

- ▶ 如果在片内RAM中调试程序，并使用了中断，那么必须将存储器映射控制设置为内部RAM模式；
- ▶ 将多个中断源设置为FIQ，将增加中断响应延时，所以建议FIQ中断只有一个中断源；
- ▶ ADS1.2规定在定义中断服务函数时必须加入关键字“__irq”，保证函数返回时会切换处理器模式；
- ▶ 在退出中断服务程序时要清零相应外设的中断标志，以及VICVectAddr寄存器，为响应下次中断作好准备。

• VIC相关启动代码分析

在LPC2100、LPC2200的启动代码中包含有VIC初始化程序，默认是禁止所有中断。

在LPC2100、LPC2200的启动代码中默认是关闭IRQ和FIQ中断的。如果用户需要使用这两个中断，则要设置CPSR寄存器的I位或F位，这需要在特权模式下处理。最简单的方法是在启动代码中设置（修改0xdf值）。

TargetResetInit函数：

```
...  
/* 初始化VIC */  
VICIntEnClr = 0xffffffff;  
VICVectAddr = 0;  
VICIntSelect = 0;  
...
```

处理器各模式堆栈设置：

```
InitStack  
  
...  
MSR   CPSR_c, #0xdf  
LDR   SP, =StackUsr  
MOV   PC, R0
```

• VIC相关启动代码分析

异常向量表:

■ 一旦产生IRQ中断，微控制器切换到IRQ模式，并跳转到向量表0x0018地址执行指令。

■ 一旦产生FIQ中断，微控制器切换到FIQ模式，并跳转到向量表0x001C地址执行指令，然后跳转到FIQ_Handler代码段。

CODE32		
AREA	vectors, CODE, READONLY	
ENTRY		
Reset		
LDR	PC, ResetAddr	
LDR	PC, UndefinedAddr	
. . .		
DCD	0xb9205f80	
LDR	PC, [PC, #-0xff0]	
LDR	PC, FIQ_Addr	
ResetAddr	DCD	ResetInit
UndefinedAddr	DCD	Undefined
. . .		
Nouse	DCD	0
IRQ_Addr	DCD	0
FIQ_Addr	DCD	FIQ_Handler